

IBM Data Engineering Course | Coursera

Big Data with Sparke & Hadoop

Summary

Module One

What is Big Data?

Introduction to Big Data

Lesson 1: What is Big Data?

Objectives

- Define **Big Data** and distinguish it from Small Data
 - Identify the **5 V's** of Big Data
 - Understand the **Big Data life cycle**
-

⌘ Key Concepts / Steps

What is Big Data?

Big Data refers to the **massive, complex, and fast-growing** sets of digital data that cannot be handled by traditional data-processing tools. It encompasses:

- **Structured, semi-structured, and unstructured** data
- Forms like text, images, video, audio, social media posts, medical records, etc.
- Generated continuously and grows **exponentially**

Small Data vs. Big Data

Feature	Small Data	Big Data
Size	Small, manageable	Massive, often measured in petabytes+
Structure	Mostly structured	Mostly semi/unstructured
Processing	Manual or light digital tools	Needs specialized frameworks (e.g., Hadoop, Spark)
Storage	Local/server-based	Cloud or distributed systems

Big Data Life Cycle

1. **Collection** – From business needs or events
2. **Storage** – Using distributed frameworks (e.g., Hadoop HDFS)
3. **Modeling** – MapReduce, data entities & relationships
4. **Processing** – Using tools like Apache Spark
5. **Visualization** – Charts/graphs to aid decision-making
6. **Feedback loop** – Data insights drive new use cases

The 5 V's of Big Data

1. **Volume** – The *amount* of data (Petabytes, Exabytes, Zettabytes)
 - Driven by: connected devices, higher-res sensors, scalable storage
2. **Velocity** – The *speed* of data generation and processing
 - Includes: batch, near real-time, streaming
3. **Variety** – The *types* and *formats* of data
 - Examples: text, audio, images, logs, APIs
4. **Veracity** – The *trustworthiness* and *quality* of data
 - Influenced by: integrity, ambiguity, completeness
5. **Value** – The *insights and outcomes* derived from Big Data
 - Helps in better decisions, resource efficiency, and innovation

Examples

- **Small Data:** Employee schedules, sports scores
- **Big Data:**
 - Social media posts (image + hashtags) = *semi-structured*
 - Medical records from millions of patients = *unstructured*
- **Data Size Fun Fact:**
 - 1 **Exabyte** $\approx$ 11 million 4K movies
 - 1 **Zettabyte** = 1 *billion* terabytes
 - 1 **Yottabyte** = 1 *trillion* terabytes 🤖

Key Takeaways

- Big Data is more than just large datasets — it's a **full lifecycle** of collecting, storing, modeling, processing, and using data.
 - The **5 V's** are essential to understanding and managing Big Data: **Volume, Velocity, Variety, Veracity, and Value**
 - Tools like **Hadoop** (for storage) and **Apache Spark** (for processing) are vital to the Big Data ecosystem.
 - Big Data is a **business enabler**, empowering faster decisions, innovation, and strategic advantage.
-

Lesson 2: Impact of Big Data

Objectives

- Identify real-world **technologies powered by Big Data**
 - Explain the **impact of Big Data** on businesses and individuals
 - Describe the **Internet of Things (IoT)** and its connection to Big Data
-

Key Concepts / Steps

Big Data in Everyday Life

Millions use Big Data daily—unknowingly!

From selfies to search queries, user-generated content fuels recommendation engines, analytics platforms, and smart assistants.

Recommendation Engines

Used by:

- **Amazon:** Products based on search, purchases, ratings, and user behavior
- **Netflix & Spotify:** Suggest content via historical preferences
- **Personal Assistants** (Siri, Alexa):
 - Use **natural language processing**
 - Interpret user voice inputs into intelligent actions
 - Learn and adapt from patterns

Predictive Features (Ex: Google Now)

- Uses **calendar, location, and device sensors**
- Forecasts needs like traffic or weather info *before* users ask
- Illustrates the **power of anticipatory Big Data analytics**

Big Data in Business

- Clive Humby: “*Data is the new oil.*”
- Geoffrey Moore: Compared data to senses like *sight and hearing*
- Big Data transforms:
 - **Decision-making**
 - **Customer experience**
 - **Market strategies**
- Drives demand for:
 - **Data scientists**
 - **Data engineers**
 - **AI/ML specialists**

Internet of Things (IoT)

- A network of **connected smart devices**
 - Includes: phones, wearables, home assistants, sensors
 - Collect and transfer data autonomously using embedded tech
- **Big Data enables IoT's true potential**

IoT Ecosystem

1. **Smart devices/sensors** collect varied data
 2. Data sent through a **Gateway**
 3. Stored in the **Cloud**
 4. Processed by **data engineers** using algorithms
 5. Translated into **actionable insights**
 - Example:
A security camera streams data; a cloud-based AI alerts on unusual activity
-

Examples

- **Amazon** recommending items based on:
 - Shopping cart contents
 - Browsing history
 - Peer customer behavior
- **Google Now:**
 - Notifies you about traffic delays or meetings without prompting
 - Blends Big Data + AI + IoT (GPS, Calendar, Internet)
- **IoT Security Camera:**
 - Constantly streams data
 - Alerts users of anomalies in real time using cloud-based analytics

Key Takeaways

- Big Data is **omnipresent**—used by people and businesses alike, often unknowingly.
 - Companies leverage Big Data to **predict behaviors**, personalize experiences, and **automate decisions**.
 - Virtual assistants and recommendation systems are direct applications of Big Data + AI.
 - **IoT** is a huge driver of Big Data growth—billions of connected devices generate data that requires advanced analytics.
 - Big Data makes IoT *useful*, and IoT fuels the volume and velocity of Big Data—it's a beautiful feedback loop.
-

■ Lesson 3: Parallel Processing, Scaling, and Data Parallelism

🧠 Objectives

- Explain **why Big Data needs parallel processing**
 - Differentiate between **linear and parallel processing**
 - Understand **horizontal scaling** and **scalability concepts**
 - Define and identify “**embarrassingly parallel**” workloads
 - Describe **fault tolerance** in distributed systems
-

✂ Key Concepts / Steps

📖 Why Parallel Processing?

- Big Data is too massive to fit or be processed efficiently on a **single computer**
 - Standard compute cycles (load → process → store) **break down** when data exceeds single-node capacity
 - Parallel processing enables **faster, more efficient, and scalable** analysis
-

🔄 Linear vs. Parallel Processing

Feature	Linear Processing	Parallel Processing
Execution	Sequential	Concurrent on multiple nodes
Speed	Slow for large/complex tasks	Fast & scalable
Error Handling	Restart whole sequence on failure	Fix locally without affecting other tasks
Suitability	Small datasets, simple logic	Big Data, high-performance computing

⚙ Parallel Processing in Action

- Instructions are split and **executed simultaneously** on multiple nodes
 - Each node handles its piece of the task with **equal processing power**
 - Benefits:
 - 🔥 Faster computation
 - 💾 Lower memory/CPU load per node
 - 🧩 Flexibility to scale in/out as needed
-

Horizontal Scaling vs. Vertical Scaling

- **Vertical Scaling (Scale-Up):**
 - Upgrade a single machine's power (RAM, CPU, etc.)
 - Hits limits quickly 🚧
 - **Horizontal Scaling (Scale-Out):**
 - Add more **nodes** (machines) of equal power
 - Creates a **computing cluster** 🖥️
 - Preferred strategy for Big Data workloads
-

“Embarrassingly Parallel” Workloads

- Tasks that can be **easily divided** and run **independently**
 - No dependency between chunks = minimal coordination
 - Example:
 - Formatting dates across partitions of a dataset
 - If one task fails, it can simply be re-run with **no impact on others**
-

Cluster Communication & Complexity

- Some computations (like **sorting**) require nodes to **coordinate state**
 - Involves **inter-process communication** or **shared file systems**
 - These are more complex but essential in real-world enterprise scenarios
-

Bringing Compute to the Data (Hadoop Style)

- In **Hadoop**, compute tasks are sent **where the data is stored** (data locality)
 - Minimizes data movement across network
 - Output is often written **on the same node** for efficiency
-

Fault Tolerance

- Systems like **HDFS** maintain **redundant copies** of data partitions
 - If a node storing partitions P1, P2, P3 fails:
 - Copies on other nodes allow **seamless recovery**
 - A new node can **rebuild lost data** from replicas
 - Hadoop's design allows **99.999% reliability** (“Five Nines”)
-



Examples

- **Embarrassingly parallel:**
 - Cleaning data across split CSV files on different nodes
 - **Fault tolerance scenario:**
 - Node with partitions crashes → new node restores data from existing replicas
 - **Horizontal scaling use case:**
 - Data size grows beyond a server's limits → add nodes instead of upgrading one giant machine
-



Key Takeaways

- **Parallel processing** is essential to handle the size and complexity of Big Data
 - It enables **concurrent execution**, which **reduces time**, cost, and resource demand
 - **Horizontal scaling** is the go-to method to scale systems for Big Data
 - “**Embarrassingly parallel**” tasks are ideal for distributed systems—simple, isolated, and fault-tolerant
 - **Fault tolerance** ensures system resilience by duplicating data across nodes and allowing **auto-recovery**
 - Big Data platforms like **Hadoop** are designed around these principles to ensure scalability, flexibility, and reliability
-



Lesson 4: Big Data Tools and Ecosystem



Objectives

By the end of this lesson, you will be able to:

- Identify the **six main categories** of Big Data tools
 - Understand the **role of each category** across the Big Data life cycle
 - Recognize key **tools and vendors** in each category
-

✂ Key Concepts / Steps



Big Data Tool Categories (The Super Six)

1. Data Technologies

2. **Analytics & Visualization**
 3. **Business Intelligence (BI)**
 4. **Cloud Providers**
 5. **NoSQL Databases**
 6. **Programming Tools**
-

1 Data Technologies

- **Purpose:** Capture, store, process, and manage Big Data efficiently
 - Handle **structured** and **unstructured** data
 - Support **parallel processing** and distributed computing
 - ✂ Tools:
 - **Apache Hadoop** – distributed storage & batch processing
 - **HDFS** – Hadoop's distributed file system
 - **Apache Spark** – real-time + batch processing
 - 🏢 Vendors: **Cloudera**, **Databricks** (support and enterprise solutions)
-

2 Analytics & Visualization Tools

- **Purpose:** Discover hidden patterns and trends; communicate them visually
 - Analytics = deep data insights
 - Visualization = graphs, charts, maps for intuitive understanding
 - ✂ Tools:
 - **Tableau**
 - **SAS**
 - **Pentaho**
 - **Palantir**
 - **Teradata**
-

3 Business Intelligence (BI)

- **Purpose:** Turn raw data into **actionable business insights**
 - Combines visualization, basic math/stats, and logical modeling
 - Used by decision-makers and executives
 - ✂ Tools:
 - **Power BI**
 - **Cognos**
 - **Oracle BI**
 - **Business Objects**
 - **Hyperion**
-

4 Cloud Service Providers

- **Purpose:** Offer **scalable infrastructure** (compute, storage, networking) + analytics as a service
 - Simplify **data ingestion, processing, and visualization**
 - Provide **on-demand** Big Data platforms
 - ✂ Platforms:
 - **AWS** (Amazon Web Services)
 - **Google Cloud Platform (GCP)**
 - **IBM Cloud**
 - **Oracle Cloud**
-

5 NoSQL Databases

- **Purpose:** Handle **large-scale, non-relational** data storage
 - Schema-less, distributed, and optimized for speed
 - Types:
 - Document-based
 - Key-value
 - Wide-column
 - Graph databases
 - ✂ Tools:
 - **MongoDB**
 - **Cassandra**
 - **CouchDB**
 - **Redis**
-

6 Programming Tools

- **Purpose:** Power the logic behind Big Data: collection, cleaning, modeling, and visualization
 - Supports **custom algorithms, ETL processes, and machine learning**
 - ✂ Languages:
 - **Python** 
 - **R**
 - **SQL**
 - **Scala**
 - **Julia**
-

Examples

- **Cloudera + Hadoop** for distributed data storage in an enterprise data lake

- **Tableau** to visualize sales trends across regions
 - **Power BI** for executive dashboards on operational performance
 - **MongoDB** for real-time app analytics
 - **Python** used to write a script that analyzes tweets for sentiment
 - **AWS** as the platform hosting your full Big Data pipeline
-

✖ Key Takeaways

- Big Data tools span **six categories**, each critical to the **data life cycle**:
 - 👉 Data technologies → collect & store
 - 👉 Analytics & viz → explore & understand
 - 👉 BI → derive decisions
 - 👉 Cloud → scale it all
 - 👉 NoSQL → handle the chaos
 - 👉 Programming tools → stitch it together
 - A mix of **open-source and commercial** tools dominate the Big Data space
 - These tools empower companies to **extract value, predict trends, and make smarter decisions** from data at scale
-

■ Lesson 5: Open Source and Big Data

🧠 Objectives

- Explain the **importance of open source** in Big Data
 - Understand **governance models** behind open source projects
 - Identify key **open-source frameworks and platforms** used in Big Data
-

✂ Key Concepts / Steps

💡 What is Open Source Software?

- Software that is **free to use, free to view, and free to modify**
- Includes both **runnable code** and the full **source code**
- Encourages **transparency, collaboration, and community-driven innovation**

🔑 True open-source = free + governed by an **open governance model**

Open Governance: The Engine Behind Openness

- Anyone can **contribute**, **suggest improvements**, and **steer development**
- Roles include:
 - **Committers** – can directly modify the code
 - **Contributors** – submit code for review
 - **Users & User Groups** – leverage and support the software
- Managed through foundations that promote **best practices** and **stability**

Open governance ≠ just visible code — it's **democratic project leadership**

Why Big Data Loves Open Source

- Big Data infrastructure requires massive, collaborative engineering
 - Open-source frameworks like **Hadoop** and **Spark** evolved *beyond* any one organization
 - Examples like **Linux** show how open-source projects can **become global standards**
-

Major Open Source Big Data Frameworks

◆ Hadoop Ecosystem (The OG Big Data Stack)

1. **MapReduce** – Batch computation model
2. **HDFS (Hadoop Distributed File System)**
 - Stores & manages massive, distributed data
 - Powers ~70% of global Big Data!
3. **YARN (Yet Another Resource Negotiator)**
 - Resource manager for apps like Hive and Spark
 - Competes with newer container-based tools like **Kubernetes**

◆ Apache Spark

- Modern alternative to MapReduce
- Supports real-time processing, ML, graph analysis, and more

◆ Apache Hive

- SQL-on-Hadoop engine
- Converts queries to MapReduce or Spark jobs

◆ Apache HBase

- NoSQL database that integrates with HDFS

- Manages storage/computation for large-scale apps

◆ Hortonworks Data Platform (HDP)

- A curated, integrated open-source Big Data stack
 - Includes: Hadoop, Hive, Spark, HBase, and more
-

Examples

- **Linux Kernel:** Born open-source, now the server OS standard 
 - **HDFS:** Stores petabytes of distributed data reliably
 - **Hive + Spark:** Used for ETL, analytics, and machine learning over Hadoop clusters
 - **HDP:** Pre-packaged Big Data ecosystem with all the essential tools
-

Key Takeaways

- **Open source is the backbone of Big Data** — offering **free, scalable, community-supported** solutions
 - Projects like **Hadoop** and **Spark** are foundational, still widely used across the industry
 - **Open governance** ensures innovation thrives beyond corporate control
 - Most Big Data tools either run on, extend, or integrate with the **Hadoop ecosystem**
 - Platforms like **HDP** make deployment and integration easier for organizations
-

Lesson 6: Beyond the Hype

Objectives

After this lesson, you'll be able to:

- Describe **true facts** about Big Data's growth and scale
 - Identify the **key sources** of Big Data
 - Differentiate between **structured, semi-structured, and unstructured** data
 - Understand the **role of cloud computing** in powering Big Data
-

✂ Key Concepts / Steps

The Explosion of Data

- **More data** was created in the last **2 years** than in all of human history
 - Global data is projected to grow by **40% per year**
 - By **2026**, we'll have created **twice as much digital data** as all previous years combined
 - Data growth measured in **zettabytes** (1 ZB = 1 billion TB!)
-

Three Main Sources of Big Data

1. **Social Data**
 - Likes, shares, tweets, videos, comments
 - Generated from platforms like Facebook, Twitter, YouTube, etc.
 2. **Machine-Generated Data**
 - Comes from **IoT sensors**, web logs, manufacturing equipment
 - Often real-time, high-frequency, large-scale
 3. **Transactional Data**
 - Created by daily operations: invoices, receipts, purchase orders, etc.
 - Found both **online and offline**
-

Three Types of Big Data

Type	Description	Examples
Structured	Predefined schema, organized in rows/columns	Databases, spreadsheets, SQL tables
Semi-Structured	Contains tags/metadata, not fully tabular	JSON, XML, emails
Unstructured	No predefined format, messy but full of insight	Images, videos, audio, social media

-  *Unstructured data = ~80% of all data in the world!*
-

Structured Data Sources

- Relational databases
- Excel or CSV files
- Easily queried using **SQL**

Semi-Structured Sources

- **JSON & XML** — use tags to represent hierarchical data
 - **Emails** — mix of metadata (To, From, Subject) + unstructured content
-

The Cloud: Big Data's Best Friend

- **Cloud Computing = Game Changer** for Big Data processing
 - Offers:
 - On-demand **compute & storage**
 - **Scalability** without upfront hardware investment
 - **Cost-efficiency** (pay only for what you use)
 - Enables rapid execution of **complex models** and handling **huge datasets**
-

Examples

- Social media APIs pulling unstructured user data from tweets
 - IoT sensors in a smart factory uploading machine data to a cloud platform
 - Email parsing tools extracting sender/subject lines into structured tables
 - Cloud platforms like **AWS** or **GCP** scaling instantly to crunch terabytes of data
-

Key Takeaways

- Big Data is growing **exponentially** and reshaping industries
 - Data comes from **social**, **machine**, and **transactional** sources
 - It exists in **structured**, **semi-structured**, and **unstructured** forms
 - **Cloud computing** makes Big Data practical, scalable, and affordable
 - The digital world is being fueled not just by data — but by **what we do with it**
-

Lesson 7: Big Data Use Cases

Objectives

After this lesson, you'll be able to:

- Identify **key industries** that heavily use Big Data
 - Explain **specific applications** of Big Data within these industries
 - Understand how companies gain a **competitive advantage** using data
-

✂ Key Concepts / Steps

Big Data = Big Decisions

- Data is being created at **unprecedented volumes**
- Industries leveraging Big Data gain **market edge** through:
 - Faster insights
 - Deeper customer understanding
 - Smarter business strategies

Top Industries Using Big Data

In order of intensity and innovation:

1. **Finance**
 2. **Technology**
 3. **Telecom**
 4. **Retail**
 5. **Government & Healthcare**
 6. **Advertising & Gaming**
 7. **Energy, Utilities, Transportation**
 8. **Consulting & Data Services**
-

Examples by Industry

Retail

- **Price Analytics:** Optimize pricing, margins, and segmentation
- **Sentiment Analysis:** Monitor social media to understand customer opinions and tailor marketing accordingly

Insurance

- **Fraud Detection:** Spot unusual claims or suspicious patterns
- **Risk Assessment:** Predict customer risk based on behavior and history

Telecom

- **Network Security:** Machine learning detects threats and applies **predictive maintenance**
- **Location-based Offers:** Real-time promotions based on GPS data
- **Traffic Forecasting:** Upgrade networks smartly based on usage patterns
- **Real-time Dynamic Pricing:** Test pricing models live with targeted groups

Manufacturing

- **Predictive Maintenance:** Sensors analyze usage and forecast failures
- **Production Optimization:** AI detects bottlenecks and proposes fixes

Automotive

- **Connected Vehicles:** Real-time sensor data powers autonomous driving decisions
- **Service Optimization:** Predictive diagnostics → timely repairs → happy customers
- **Insurance and Traffic Analytics:** Influence policy rates and improve navigation

Finance

- **Fraud Detection & Risk Scoring:** Just like insurance
- **Customer Segmentation:** Group users by demographics, habits, or interactions
- **Algorithmic Trading:** ML models make split-second trading decisions

Key Takeaways

- **Data is the new currency**, and every major industry is investing in it
- Big Data drives:
 - ✓ Efficiency
 - ✓ Innovation
 - ✓ Customer satisfaction
- Each industry applies Big Data differently, but the goal is the same: **Smarter decisions. Bigger impact. Faster wins.**
- Industries like **finance, telecom, and retail** are front-runners — but Big Data's reach is now **truly universal**

Lesson 8: Introduction to Emerging Big Data Technologies

Objectives

- Explain the purpose and capabilities of **Pulsar, Druid, and PrestoDB**
- Differentiate between **traditional** (Hadoop, Spark) and **emerging** big data tools
- Match each technology with the right use case (e.g., real-time streaming vs. querying vs. analytics)
- Evaluate **scalability, latency, and performance** trade-offs

- Choose the most suitable technology stack for modern data applications
-

✂ Key Concepts / Steps

◆ Apache Pulsar

- **Messaging + Streaming** in one tool
- Built for **high-throughput + low-latency**
- Supports **multi-tenancy**, **geo-replication**, and **topic retention**
- Decouples **compute** from **storage** using **Apache BookKeeper**

◆ Apache Druid

- **Real-time analytics** engine optimized for **time-series** data
- Ingests from both batch + streaming sources
- Uses **columnar storage** for fast OLAP queries
- Horizontal scalability for **interactive dashboards** and **fraud detection**

◆ PrestoDB

- **Distributed SQL engine** for querying across multiple data sources (HDFS, S3, NoSQL, MySQL...)
 - Supports **federated queries** without ETL
 - Designed for **ad hoc analysis**, **BI dashboards**, and **data lakes**
 - Horizontal scalability; interactive speeds for big queries
-

Examples

-  **Retail:** Use Presto to join sales data from MySQL with user activity in S3.
 -  **Monitoring dashboards:** Use Druid to show real-time metrics from IoT sensors.
 -  **Microservices architecture:** Use Pulsar to queue and stream events between services.
 -  **ETL + ML workflows:** Combine Spark for ML, Pulsar for data streaming, and Druid for dashboards.
-

Key Takeaways

- **Pulsar** is best for **event-driven** systems and real-time **streaming pipelines**
- **Druid** excels in **real-time analytics** and **OLAP** over time-series data
- **PrestoDB** is ideal for **interactive querying** across **heterogeneous data sources**

- Compared to **Hadoop** and **Spark**, these tools offer **lower latency**, **greater flexibility**, and **more targeted use cases**
 - Selecting the right tool depends on your **latency**, **scalability**, and **data source diversity** needs
-

Lesson's Summary

- Personal assistants like Siri, Alexa, and Google Now use Big Data and IoT to gather data and devise answers.
 - Big Data Analytics helps companies gain insights from the data collected by IoT devices.
 - Big Data requires parallel processing on account of massive volumes of data that are too large to fit on any one computer.
 - “Embarrassingly parallel” calculations are the kinds of workloads that can easily be divided and run independently of one another. If any single process fails, that process has no impact on the other processes and can simply be re-run.
 - Open-source projects, which are free and completely transparent, run the world of Big Data and include the Hadoop project and big data tools like Apache Hive and Apache Spark.
 - The Big Data tool ecosystem includes the following six main tooling categories: data technologies, analytics and visualization, business intelligence, cloud providers, NoSQL databases, and programming tools.
-

Module 1 – Glossary: What is Big Data?

Objectives

- Understand key terms used across the Big Data landscape.
 - Familiarize yourself with core technologies, architectures, and tools.
 - Recognize terminology crucial for industry use and certification exams.
-

Key Terms & Definitions (Grouped for Clarity)

Foundational Concepts

Term	Definition
Big Data	Extremely large datasets with high volume, velocity, variety (and veracity) that exceed traditional processing tools.
Big Data Analytics	Applying advanced techniques to analyze structured, semi-structured, and unstructured data to gain business insights.
Velocity / Volume / Variety / Veracity	The four "V"s that define the core characteristics of Big Data.
Transactional Data	Data generated from day-to-day business activities (e.g., invoices, payments).
Social Data	Data from social platforms (likes, tweets, videos, comments).
Machine Data	Data from sensors, logs, and IoT devices.

Technologies & Frameworks

Term	Definition
Hadoop	Open-source framework for distributed processing of big data using simple programming models.
MapReduce	Hadoop's original processing engine; divides work across clusters (Map → Reduce).
Apache Spark	In-memory data processing framework for fast, iterative analytics and machine learning.
YARN	Resource manager in Hadoop, manages and schedules processing jobs.
Apache HBase	A scalable NoSQL database often used with Hadoop for real-time read/write access.
Hive	A data warehouse built on Hadoop that supports SQL-like querying.
HDFS	Hadoop's distributed storage layer; breaks files into blocks with fault tolerance.

Cloud & Infrastructure

Term	Definition
Cloud Computing	On-demand delivery of computing services via the internet.
Cloud Providers	Companies offering infrastructure and analytics services (AWS, IBM, GCP, Oracle).

Term	Definition
ETL (Extract, Transform, Load)	Process of preparing data for analytics by moving it into warehouses or lakes.

Tools & Languages

Term	Definition
Big Data Programming Tools	Languages used for data tasks: Python, R, SQL, Scala, Julia.
Business Intelligence (BI)	Techniques and tools to transform data into actionable insights.
NoSQL Databases	Flexible databases for large-scale data: MongoDB, Cassandra, Redis, CouchDB.

Open Source Ecosystem

Term	Definition
Open-source Software	Software with freely accessible and modifiable source code.
Committer	A person in an open-source project who has write access to make code changes.

Business & Analytics Terms

Term	Definition
Price Analytics	Using data to identify optimal pricing strategies and maximize margins.
Sentiment Analysis	Analyzing customer opinions using social data to inform marketing and product decisions.
Relational Databases	Traditional databases with structured tables and defined relationships.

Key Takeaways

- Big Data is more than size—it's about speed, variety, and complexity.
 - Hadoop, Spark, and cloud computing form the technological core.
 - BI, ETL, and programming tools are enablers of insight generation.
 - Understanding this glossary is foundational for deeper learning in data engineering and analytics.
-

Module two

Introduction to the Hadoop Ecosystem

Introduction to Hadoop

Lesson 1: Introduction to Hadoop

Objectives

- Define Hadoop and explain its history
 - Understand why Hadoop emerged as a Big Data solution
 - Identify core components of Hadoop
 - Describe key challenges and limitations of Hadoop
-

What Is Hadoop?

Hadoop is an **open-source framework** for processing **massive datasets** across **distributed computing systems**.

- Not a database, but a **Big Data ecosystem**
 - Supports:
 -  **Structured data** (e.g., tabular sales records)
 -  **Unstructured data** (e.g., social media posts, sensor logs)
 -  **Semi-structured data** (e.g., JSON, XML)
 - Works by running applications on **clusters** of inexpensive machines (commodity hardware)
-

Hadoop: A Brief History

Year	Milestone
1999	Apache Software Foundation founded
2002	Doug Cutting & Mike Cafarella create Nutch (web search engine)
2006	Doug joins Yahoo; project split into crawler + distributed processing (Hadoop)
2008	Yahoo releases Hadoop to Apache

Core Components of Hadoop

Component	Description
Hadoop Common	Utility libraries used across all Hadoop modules
HDFS (Hadoop Distributed File System)	Storage system that splits data into blocks and distributes them

Component	Description
MapReduce	Processing model that breaks tasks into parallel jobs
YARN (Yet Another Resource Negotiator)	Manages resources (CPU, RAM) across the Hadoop cluster

⚠ Note: Hadoop supports **batch processing** and is not ideal for **low-latency** tasks.

💡 Why Use Hadoop?

- When data grows from **gigabytes** → **terabytes** → **petabytes**, traditional systems fail.
 - Hadoop solves this by:
 - 🔄 Scalable architecture (horizontal scaling)
 - 💰 Cost-effective (runs on commodity hardware)
 - 🧠 Efficient for parallel tasks
-

⚠ Hadoop Limitations

- ❌ Poor for random access or interactive queries
 - ❌ Struggles with dependencies in data (e.g., record 1 depends on record 2)
 - ❌ Not good with small files (lots of them)
 - ❌ High latency (not suitable for real-time applications)
 - ❌ Inefficient for complex computations on small data
-

🔧 Tools Built on Top of Hadoop

To address Hadoop's limitations, tools like:

- **Hive:** SQL-like querying for analytics
 - **Pig:** Supports multi-query optimization and batch execution
-

✓ Key Takeaways

- Hadoop is a foundational open-source framework for Big Data.
 - It includes **HDFS**, **MapReduce**, **YARN**, and **Hadoop Common**.
 - Strengths: Scalability, fault tolerance, batch processing.
 - Weaknesses: Latency, random access, dependency handling, small files.
 - New tools like Hive & Pig improved developer experience on Hadoop.
-

📘 Lesson 2: introduction to MapReduce

🎯 Objectives

By the end of this lesson, you should be able to:

- Define **MapReduce** and its role in Hadoop
 - Explain the **Map** and **Reduce** phases
 - List key **components** of MapReduce
 - Recognize **real-world use cases** of MapReduce in various industries
-

🧠 What Is MapReduce?

🧩 **MapReduce** is the **processing engine** in Hadoop.

It is a **distributed computing** framework designed to **scale** across hundreds or thousands of servers and handle **large-scale data processing**.

- Built for **parallel processing** across nodes in a cluster
 - Originally developed using **Java**, but also supports C++, Python, Ruby, R, etc.
 - Works by splitting data and performing tasks **independently and simultaneously**
-

🔗 Core Concepts: Map + Reduce

📖 Map Phase

- Takes raw input (e.g., a file in HDFS)
- Breaks it into smaller chunks (splits)
- Converts data into **key-value pairs**
e.g., "Teju" → (Teju, 1)
- Output: preliminary data organized for processing

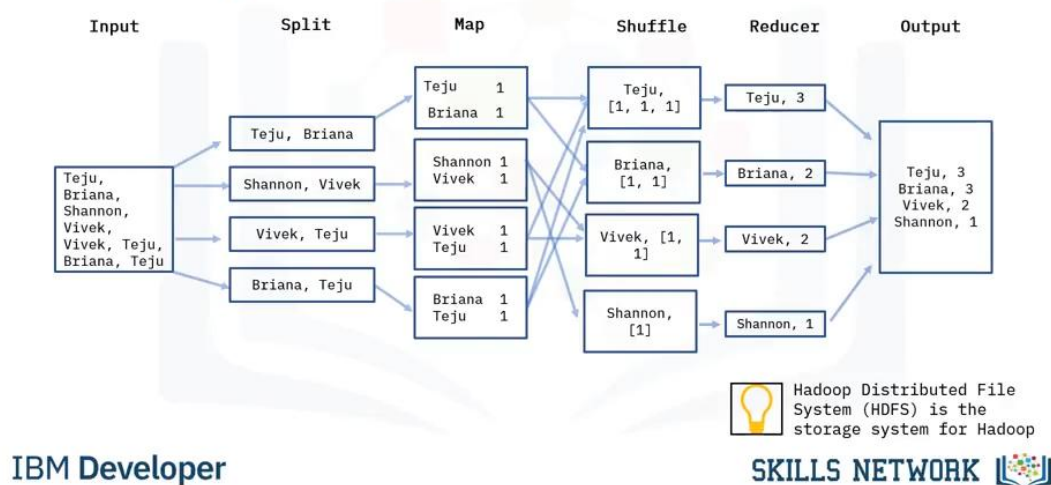
🔄 Shuffle + Sort (Intermediate)

- Groups the same **keys** together
e.g., Teju $\rightarrow$ [1, 1, 1]
- Prepares the data for the Reduce phase

+ Reduce Phase

- Aggregates the values associated with each key
e.g., Teju $\rightarrow$ 3
- Produces final result
- Writes output back to **HDFS**

How MapReduce works



Hadoop Node Structure

Node Type	Description
Name Node	Manages metadata (directory structure, file locations)
Data Node	Stores and processes actual data blocks

 Think of MapReduce like a factory:

The **Map** workers sort and label packages, the **Shuffle** team organizes them by destination, and the **Reduce** workers seal and count the boxes for delivery.

Advantages of MapReduce

-  **Highly scalable** across distributed systems
-  Enables **parallel processing**, saving time
-  Handles **structured, unstructured, and semi-structured** data

-  Supports multiple programming languages
 -  Works with huge datasets across **commodity hardware**
-

Use Cases

Industry	Application
Social Media	Track profile views and post engagement (e.g., Instagram, LinkedIn)
Entertainment	Recommend content based on user behavior (e.g., Netflix)
Finance	Detect fraud or suspicious transactions
Advertising	Analyze ad engagement and behavior (e.g., Google Ads)

Summary

- **MapReduce** is a two-phase processing model:
 - **Map** transforms input into key-value pairs
 - **Reduce** aggregates and finalizes the results
 - It's central to the **Hadoop ecosystem**
 - Useful across industries for scalable, parallel data processing
 - Best suited for **batch jobs** and **massive datasets**
-

Lesson 3: Hadoop Ecosystem

Objectives

- Identify the **four stages** of the Hadoop ecosystem
 - Differentiate between **core** and **extended** components
 - Recognize popular **tools** used in each stage
-

What Is the Hadoop Ecosystem?

The **Hadoop Ecosystem** is not just a framework—it's an entire **modular system** of tools and components that work together to ingest, store, process, analyze, and access **Big Data**.

-  **Core components:** Fundamental to Hadoop's operation
 -  **Extended components:** Additional libraries and tools used to enhance and specialize Hadoop capabilities
-

Core Hadoop Components

Component	Role
HDFS (Hadoop Distributed File System)	Stores distributed data across nodes
MapReduce	Processes large-scale data in parallel
YARN (Yet Another Resource Negotiator)	Manages resources across the cluster
Hadoop Common	Utility libraries and configuration used by other modules

Stages of the Hadoop Ecosystem

Let's break the Hadoop ecosystem into **4 key stages** with tools for each:

1 Ingesting Data

Tool	Description
Flume	Collects & streams real-time log data into Hadoop
Sqoop	Transfers bulk data from relational databases to Hadoop

 *Analogy*: Flume = real-time firehose, Sqoop = batch pipeline from a traditional database

2 Storing Data

Tool	Description
HDFS	File-based distributed storage
HBase	Column-based NoSQL database on top of HDFS
Cassandra	NoSQL database with no single point of failure

Note: HBase supports **random access**, unlike HDFS (which is optimized for sequential reads).

3 Processing & Analyzing Data

Tool	Description
Pig	Procedural scripting language (like Python) for analyzing data
Hive	SQL-like, declarative tool for generating reports and queries



Pig = Write data flows step-by-step.



Hive = Ask what you want, let the engine figure out how.



Accessing Data

Tool

Description

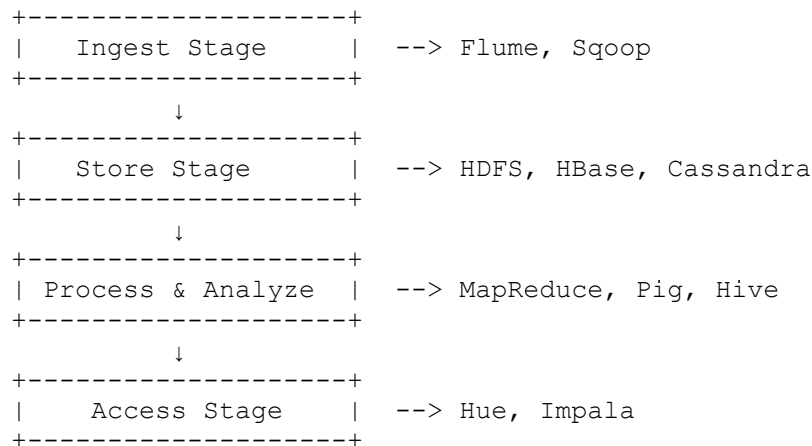
Impala SQL engine for quick querying and interactive access

Hue Web UI to explore, query, and manage Hadoop tools with ease

Hue = Friendly dashboard for developers and analysts.



Interactions Between Stages



Each stage builds on the previous one, allowing data to flow smoothly from raw form to refined business insights.



Summary

- Hadoop Ecosystem consists of **Core** (HDFS, MapReduce, YARN, Common) and **Extended** tools.
- Four major **stages**:
 1. **Ingest** → Flume, Sqoop
 2. **Store** → HDFS, HBase, Cassandra
 3. **Process & Analyze** → MapReduce, Pig, Hive
 4. **Access** → Hue, Impala
- Each stage plays a critical role in building a **complete Big Data pipeline**.

Lesson 4: HDFS

Objectives

- Define **HDFS** and its **key features**
 - Understand the **HDFS architecture** and core concepts like **blocks**, **replication**, and **rack awareness**
 - Describe **read and write operations** in HDFS
-

What Is HDFS?

HDFS (Hadoop Distributed File System) is the **storage layer** of Hadoop designed to store **massive datasets** across **multiple machines**.

 It works by:

- **Splitting** files into **blocks**
- **Replicating** those blocks
- Storing them on **distributed data nodes**

It is **optimized** for **streaming large files** and **high fault-tolerance** on **low-cost hardware**.

Key Features of HDFS

Feature	Description
 Cost Efficiency	Uses commodity hardware , reducing infrastructure costs
 Scalability	Can scale up to hundreds or thousands of nodes
 Large Storage	Handles petabytes of data in structured, semi-structured, and unstructured formats
 Replication	Ensures data availability even if nodes fail
 Portability	Easily deployed on different platforms
 Write-once, Read-many	Files cannot be edited after write, but appending is allowed

Concept: HDFS Block

- A **block** is the **smallest unit** of storage in HDFS.
- Typical block size: **64 MB** or **128 MB** (configurable)
- A **500 MB** file with 128 MB blocks → 4 blocks (128, 128, 128, 116 MB)

HDFS Architecture

HDFS uses a **Primary/Secondary Node model**:

Node Type	Description
Name Node (Primary)	Controls file system namespace, manages block metadata, authorizes access
Data Node (Secondary)	Stores the actual data blocks and handles read/write requests

Each **cluster** contains:

- 1 **Name Node**
- **Many Data Nodes**

Rack Awareness

- A **rack** is a group of ~40–50 data nodes connected to the same switch
- The **Name Node** uses rack IDs to:
 - Place replicas on **different racks** (for redundancy)
 - Choose the **closest data node** for performance
- Reduces **network traffic** and improves **throughput**

Data Replication in HDFS

Concept	Description
 Replication Factor	Number of copies of each block (default is 3)
 Placement Strategy	One copy on local rack, others on separate racks
 Fault Tolerance	Enables data recovery when nodes/racks fail

Example:

For a 500 MB file split into 4 blocks and **replication factor = 2** →
8 total block copies spread across different racks.

Read Operation in HDFS

1. **Client** → **Name Node**: Requests file access
2. **Name Node**: Authenticates and returns block locations

3. **Client** → **Data Node(s)**: Reads data via `FSDataInputStream`
4. **Client** → **close()**: Ends the session

✅ The Name Node optimizes performance by choosing **nearest** data nodes.

Write Operation in HDFS

1. **Client** → **Name Node**: Requests to create a new file
2. **Name Node**:
 - Validates permissions
 - Verifies file doesn't already exist
3. **Client** → **Data Nodes**: Starts writing data
4. **Data Nodes**:
 - Store blocks
 - Replicate as configured
 - Send confirmation to the client

❌ If file already exists → **I/O Exception** raised.

Summary

Topic	Key Points
HDFS Purpose	Storage layer for Big Data in Hadoop
Storage Strategy	Splits data into blocks , stores across data nodes
Architecture	One Name Node + many Data Nodes
Read/Write Model	Write-once, read-many
Replication	Ensures fault tolerance and high availability
Rack Awareness	Improves performance and redundancy
Interface	Operated via command-line tools or API

Lesson 5: Hive

Objectives

- Understand the **purpose of Hive** and why it's used
 - Compare **Hive vs. traditional RDBMS**
 - Identify the **key features and architecture** of Hive
 - Explain important **concepts** like partitioning, HiveQL, and metadata storage
-

What Is Hive?

Apache Hive is a **data warehouse software** built on top of Hadoop, designed for:

- **Reading, writing, and managing** large **tabular datasets**
- Supporting **SQL-like queries** through **HiveQL**
- Processing **structured and semi-structured data**
- Scaling to **petabyte-level data volumes**

 Use case: Hive is ideal for **data analysis, reporting, and batch processing**.

Hive vs. Traditional RDBMS

Feature	Hive	Traditional RDBMS
 Purpose	Data warehouse	Database management
 Query Language	HiveQL (SQL-like)	SQL
 Data Access	Write once, read many	Read and write anytime
 Use Case	Batch/static data analysis	Real-time processing
 Scale	Petabytes	Usually terabytes
 Schema	Not enforced before load	Schema must match before load
 Partitioning	Supported natively	Not always supported
 Storage	HDFS	Local/RDBMS disk

Key Features of Hive

Feature	Description
 SQL Familiarity	Uses HiveQL , which is similar to SQL
 Data Cleaning	Supports filtering and transformation
 File Format Support	Text, Sequence files, RCFiles
 Partitioning	Breaks tables by column values (e.g., date, city)
 Scalable	Designed for massive datasets
 Interoperability	Connects via JDBC/ODBC clients

Hive Architecture Overview

Hive is structured in **three main layers**:

Layer	Components	Description
1. Hive Client	JDBC, ODBC, CLI	Interfaces for user input and application integration
2. Hive Services	HiveServer, Compiler, Optimizer, Executor, Driver	Manages query parsing, optimization, execution
3. Hive Storage & Compute	Meta Store, HDFS, Hadoop jobs	Where data resides and processing happens

Components in Detail

Hive Client

- **JDBC Client:** For Java applications
- **ODBC Client:** For non-Java apps
- **CLI:** Command-line interface for direct HiveQL queries

Hive Services

- **Hive Server:** Accepts queries from clients
- **Driver:** Manages query lifecycle, sessions, and coordinates components
- **Compiler:** Converts HiveQL to MapReduce (or other execution engines)
- **Optimizer:** Improves query performance, splits jobs
- **Executor:** Runs the actual query jobs on Hadoop

Metadata & Storage

- **Meta Store:** Stores **schema, table names, partitions**, etc.
 - **Storage:** Hive stores data in **HDFS** and metadata in **RDBMS**
-

Hive Concepts Summary

Concept	Description
Data Warehouse	Stores historical, analytical data
Partitioning	Divides table for faster querying
Write Once, Read Many	Optimized for batch operations
HiveQL	SQL-like query language for Hive
Meta Store	Central schema storage
File Format Support	Text, Sequence, RCFile

✓ Summary

- Hive is built for **big data analysis**, supporting **SQL-like queries** over **HDFS-stored data**
 - Unlike RDBMS, Hive is optimized for **read-heavy, batch processing** workloads
 - Hive architecture includes **Clients, Services, and Storage/Compute layers**
 - **Partitioning, scalability, and schema-on-read** are key advantages of Hive
-

📘 Lesson 6: HBASE

🎯 Objectives

- Define **HBase** and understand its role in Big Data
 - Compare **HBase vs. HDFS**
 - List **HBase features and use cases**
 - Describe the **HBase architecture and components**
 - Understand **column families, regions**, and other key HBase concepts
-

📌 What Is HBase?

Apache HBase is a:

- **Column-oriented, non-relational (NoSQL)** database
- Built on top of **HDFS** for **real-time read/write access** to big datasets
- Suitable for **sparse data, write-heavy**, and **low-latency** applications

🎯 Common use cases include:

- Real-time sensor data logging (e.g., health monitors)
 - Time-series databases
 - Messaging platforms
 - Search and log analytics
-

🔧 Key Features of HBase

Feature	Description
🗄️ Column-Oriented	Data organized in column families (flexible schema)
⚡ Real-Time Access	Supports random reads and writes
🔄 Dynamic Updates	Can insert, update, delete at record-level

Feature	Description
 Linear Scalability	Easily scalable by adding region servers
 Fault Tolerant	Built-in fault tolerance through HDFS
 Replication	Supports data replication across clusters
 Java API	Easy integration with Java applications

Column Family Structure (Example)

Table: Patient_Logs

Column Family	Column Qualifier	Value
patient_details	name / age	"Alice" / 32
vitals	heart_rate	87 bpm
timestamps	log_time	2025-07-14 09:00

✓ All columns must belong to a **predefined column family**, but new columns can be added at runtime within those families.

HBase vs HDFS

Aspect	HBase	HDFS
 Data Model	Table with rows and columns (column families)	Flat files
 Function	Database-like (store + query + update)	Storage-only
 Operations	CRUD (Create, Read, Update, Delete)	Write-once, read-many
 Access	Real-time, random access	Batch access
 Flexibility	Dynamic schema	Static file system
 Use Case	Apps requiring fast read/write (e.g., analytics, logs)	Data lake, archival

HBase Architecture

1. HMaster

- Manages region servers
- Assigns regions and handles schema/metadata operations

2. RegionServer

- Handles client requests (read/write)
- Manages **regions** (table partitions)
- Communicates **directly with clients** (bypassing HMaster for efficiency)

3. Region

- Smallest unit of distribution
- Contains:
 - **MemStore** (in-memory writes)
 - **HFile** (disk storage)
- Each region corresponds to a **subset of a table**

4. ZooKeeper

- **Coordination service**
- Manages **node health, leader election, and synchronization**

5. HDFS

- **Underlying storage layer**
- Provides replication and durability

💡 Key Concepts

Term	Definition
 Column Family	Logical grouping of columns (defined during schema creation)
 Schema Flexibility	Columns within families can be added at any time
 Real-Time Access	Ideal for quick reads/writes at row level
 Fault Tolerance	Achieved via HDFS replication and ZooKeeper management
 Row Key	Unique identifier for each row, enables fast lookups

✅ Summary

- **HBase** is a **real-time, columnar, NoSQL database** that works on **HDFS**
 - Designed for **high-speed read/write operations** over massive datasets
 - HBase schema is **defined by column families**, not fixed columns
 - Major components: **HMaster, RegionServer, Region, ZooKeeper, HDFS**
 - Differentiated from HDFS by its ability to **dynamically update and query structured data**
-

Lesson's Summary:

- Hadoop is an open-source framework for Big Data that faces challenges when encountering dependencies and low-level latency.
- MapReduce, a parallel computing framework used in parallel computing, is flexible for all data types, addresses parallel processing needs for multiple industries, and contains two major tasks, “map” and “reduce.”
- The four main stages of the Hadoop Ecosystem are ingest, store, process and analyze, and access.
- Key HDFS benefits include its cost efficiency, scalability, data storage expansion, and data replication capabilities. Rack awareness helps reduce the network traffic and improve cluster performance. HDFS enables “write once, read many” operations.
- Suited for static data analysis and built to handle petabytes of data, Hive is a data warehouse software for reading, writing, and managing datasets.
- Hive is based on the “write once, read many” methodology, doesn’t enforce the schema to verify loading data, and has built-in partitioning support.
- Linearly scalable and highly efficient, HBase is a column-oriented nonrelational database management system that runs on HDFS and provides an easy-to-use Java API for client access.
- The HBase architecture consists of HMaster, Region servers, Region, Zookeeper, and HDFS. A key difference between HDFS and HBase is that HBase allows dynamic changes compared to the rigid architecture of HDFS.

Hadoop Ecosystem Commands & Tools (Cheat Sheet)

⌘ General Hadoop CLI

Command	Description	Example
<code>bin/hadoop</code>	Runs Hadoop commands. No args shows all available commands.	<code>bin/hadoop</code>

File Operations (Linux Shell)

Command	Purpose	Example
cat	Read and output file contents.	cat file1.txt file2.txt
cd	Change directory.	cd /usr/local/folder1, cd -, cd ..
mkdir	Make a directory.	mkdir myfolder
ls	List contents of a directory.	ls, ls -ltr, ls -a
tar	Archive files/directories.	tar -cvf archive.tar /dir
wget	Download files non-interactively.	wget http://example.com/file.txt

HDFS (Hadoop Distributed File System)

Command	Function	Example
hdfs dfs -ls /	List root directory contents in HDFS.	
hdfs dfs -mkdir /path	Make directory in HDFS.	hdfs dfs -mkdir /user/username/mydirectory
hdfs dfs -put	Upload from local to HDFS.	hdfs dfs -put localfile.txt /user/username/
hdfs dfs -cat	Display file contents in HDFS.	hdfs dfs -cat /user/username/file.txt

Hive (Data Warehousing)

Command	Purpose	Example
CREATE DATABASE	Create a new DB.	CREATE DATABASE your_database;
USE	Select database.	USE your_database;
CREATE TABLE	Define table with schema.	see full example below 
LOAD DATA INPATH	Load data into Hive table.	LOAD DATA INPATH '/user/username/hdfsfile.txt' INTO TABLE mytable;
SHOW TABLES	List tables in DB.	SHOW TABLES;
SELECT * FROM	View all rows in table.	SELECT * FROM tablename;

Example Hive Table Creation

```
CREATE TABLE employees (  
    id INT,  
    first_name STRING,  
    last_name STRING,  
    email STRING,  
    hire_date DATE  
)  
ROW FORMAT DELIMITED  
FIELDS TERMINATED BY ','  
STORED AS TEXTFILE;
```

Docker & DevOps

Command	Purpose	Example
<code>docker exec</code>	Run a command inside running container.	<code>docker exec -it container_name ls /app</code>
<code>docker pull</code>	Download image.	<code>docker pull ubuntu:latest</code>
<code>docker run</code>	Run image as container.	<code>docker run ubuntu echo hello</code>
<code>docker-compose up</code>	Start multi-container app from YAML config.	<code>docker-compose up</code>

Sample docker-compose.yml

```
version: '3'  
services:  
  web:  
    image: nginx:latest  
    ports:  
      - "80:80"  
  db:  
    image: postgres:latest  
    environment:  
      POSTGRES_PASSWORD: example_password
```

Network Tools

Command	Function	Example
<code>curl</code>	Send HTTP requests.	
- GET Request		<code>curl https://www.example.com</code>
- POST Request		<code>curl -X POST -d "name=John&age=30" https://www.example.com/api</code>



Command	Function	Example
<code>git clone</code>	Clone a repo.	<code>git clone https://github.com/example/repo.git</code>

Big Data Fundamentals

Term	Definition
Big data	Data sets too large or complex for traditional databases to handle efficiently, characterized by Volume, Velocity, and Variety.
Big data analytics	Applying advanced analytical techniques to vast and diverse data sets (structured, semi-structured, unstructured).
Data sets	Collections of data used frequently, often powering dashboards and analytics.
Structured data	Well-organized data (e.g., SQL tables) easily processed by machines.
Unstructured data	Data without a predefined model (e.g., images, videos, social media).
Semi-structured data	Data with partial organization (e.g., JSON, XML, CSV).
Low latency data access	Fast data retrieval with minimal delay; essential for real-time applications.
High throughput	High rate of successful data processing, critical for performance.
Data warehouse	Centralized repository that stores historical data for analysis.

Hadoop & Distributed Computing

Term	Definition
Hadoop	An open-source framework for distributed processing of large data sets.
Hadoop Common	Core utilities and libraries used by other Hadoop modules.
HDFS (Hadoop Distributed File System)	Storage layer of Hadoop that splits files into blocks and stores them across nodes.
MapReduce	Java-based model for processing large data sets via mapping and reducing operations.
YARN	Resource management layer in Hadoop that handles scheduling and job execution.
Hadoop Ecosystem	Includes tools and services like HDFS, MapReduce, Hive, Pig, Sqoop, Flume, etc.
Extended Hadoop Ecosystem	Add-on libraries commonly used with Hadoop, e.g., HBase, Hue, Oozie.

Term	Definition
Hadoop Ecosystem stages	Ingest → Store → Process → Analyze → Access

System Architecture & Components

Term	Definition
Cluster	A group of machines (nodes) that work together in parallel.
Node	A single machine in the cluster.
Primary node (NameNode)	Manages the file system namespace and regulates client access.
Secondary node (DataNode)	Stores and retrieves blocks as instructed by the NameNode.
Rack	A group of 40–50 nodes sharing a network switch.
Rack awareness	Optimizing data placement based on network proximity.
Replication / Replication factor	Copying data across multiple nodes/racks for fault tolerance.
Fault tolerance	The system's ability to continue operations despite failures.
File system	Directory structure managing data storage.
Streaming	Transferring data at a consistent rate.

Data Movement & Ingestion

Term	Definition
Data ingestion	The process of importing and loading data into storage or analytics systems.
Sqoop	Tool to transfer data between RDBMS and Hadoop.
Flume	Distributed service for collecting and moving large volumes of log data into HDFS.
InputSplits	Divides input data into logical splits for MapReduce processing.
Shuffle	Transfers intermediate outputs from mappers to reducers during MapReduce.

Querying & Analytics Tools

Term	Definition
Hive	SQL-like data warehouse layer for Hadoop.
Hive client / JDBC / ODBC	Interfaces to connect external applications to Hive.

Term	Definition
Hive server	Executes queries and handles client requests.
Hive services	Includes the driver, CLI, Web UI, etc.
Hive Web Interface	GUI to interact with Hive (browse, query, manage).
Meta store	Stores metadata about Hive tables and schemas.
Schema	Logical grouping of named objects (tables, views, etc.).
Partitioning	Dividing tables into parts by column values for efficiency.

Other Hadoop Ecosystem Components

Term	Definition
Pig	A high-level platform for analyzing large datasets using a scripting language (Pig Latin).
Impala	Query engine for real-time interactive SQL on Hadoop.
Hue	Web-based UI for Hadoop ecosystem components.
HBase	Columnar NoSQL database running on top of HDFS.
HMaster / Region / Region Server	Core components in HBase managing data distribution and region assignments.

Programming Concepts & Internals

Term	Definition
Map / Reduce	Map converts input to key-value pairs; Reduce aggregates results.
Driver	Receives queries and sends them to the compiler in Hive.
Executor	Executes tasks in a distributed computation job.
Optimizer	Improves efficiency by transforming execution plans.
Parser	Translates query input into internal format.

Tools, Interfaces, & Support Systems

Term	Definition
CLI (Command-Line Interface)	Text-based way to interact with the system.
Apache	Foundation for open-source software like Hadoop, Hive, Pig.
Apache ZooKeeper	Maintains configuration and coordination between distributed systems.
Apache Nutch	Scalable web crawler.

Term	Definition
Apache Cassandra	Fault-tolerant, NoSQL database without a single point of failure.
Commodity hardware	Inexpensive, widely available computing equipment.

Bonus Terms

Term	Definition
Anomaly detection	Identifying unusual patterns in time-series or large datasets.
Distributed computing	Splitting workloads across multiple machines working together.
Parallel computing	Executing multiple calculations simultaneously.
Relational DB / RDBMS	Structured data systems using SQL.

Module Three

Apache Spark

Introduction to Apache Spark

Lesson 1: Why use Apache Spark

Objectives

- Describe **Apache Spark's** key features
 - Understand **distributed computing** and how it differs from parallel computing
 - Explain the **benefits of distributed computing and Spark**
 - Compare **Apache Spark vs. MapReduce**
-

✂ Key Concepts / Steps

- **Apache Spark is:**
 - Open-source (under the Apache Foundation)
 - An **in-memory** distributed data processing framework
 - Ideal for **iterative analysis** on **massive datasets**
 - Written primarily in **Scala**, but supports **Python, Java**, and **R** through APIs
- **Distributed Computing:**
 - A system where **multiple computers** (nodes) work together on a task
 - Often confused with parallel computing, but:

- **Parallel computing** shares memory
 - **Distributed computing**: each node has **its own memory**
 - **Advantages of Distributed Computing**:
 1. **Scalability**: Easily scale horizontally by adding machines
 2. **Fault Tolerance & Redundancy**: System continues even if one or more machines fail
 - **Why Spark?**
 - Leverages all distributed computing advantages
 - Enables **parallel in-memory data processing**
 - Offers a **unified framework** for:
 - Data engineering (via Spark Core, SparkSQL, DataFrames)
 - Data science (via SparkML for ML and Spark Streaming for real-time data)
 - **Spark vs. MapReduce**:
 - **MapReduce** reads/writes to disk between each iteration — slow & expensive
 - **Spark** processes data **in-memory**, drastically improving performance (faster by **orders of magnitude**)
-

Examples

- A Spark cluster with 8 machines can continue operating even if several machines crash — thanks to redundancy.
 - Data scientists can use **SparkML** to run machine learning models at scale, while engineers use **SparkSQL** for fast querying.
-

Key Takeaways

- Spark is **fast, scalable, fault-tolerant**, and supports both **engineering** and **data science** workflows
 - It outperforms traditional tools like **MapReduce** by minimizing disk I/O
 - **Distributed computing** enables robust, scalable systems — and Spark brings it to life with a flexible programming model
-

■ Lesson 2: Functional Programming Basics

🎯 Objectives

- Explain **Functional Programming (FP)**
 - Understand **Lambda functions** and their use
 - Relate **Functional Programming to Apache Spark**
-

■ Key Concepts

◆ What is Functional Programming (FP)?

- A **declarative programming style** that emphasizes “**what to solve**” instead of “**how to solve**”
- Inspired by **mathematical functions** like $f(x) = x + 1$
- FP avoids step-by-step (imperative) instructions and instead focuses on expressions and transformations

◆ Expressions vs. Instructions

- **Declarative (FP):**
- `list.map(lambda x: x + 1)`

Emphasis: What should happen

- **Imperative:**
- `for i in range(len(list)):`
- `list[i] += 1`

Emphasis: How it should happen

◆ Historical Roots

- First FP language: **LISP** (1950s)
 - Modern FP languages: **Scala, Python, R, Java**
 - **Scala** is used heavily in **Apache Spark** due to its functional programming strengths
-

◆ Lambda Calculus & Lambda Functions

- Based on **lambda calculus**, a mathematical foundation where **every computation is a function**

- **Lambda functions:** Anonymous functions used to perform computations on-the-fly
 - Python: `lambda x, y: x + y`
 - Scala: `(x: Int, y: Int) => x + y`
-

◆ Benefits of FP in Big Data (esp. Spark)

- FP enables **parallelism** by design:
 - Break tasks into chunks
 - Run in parallel across nodes without changing the function code
 - Great for **scalability**:
 - 1 KB or 1 PB — Spark handles both using the same logic
 - Just add more **compute resources**, and Spark distributes tasks via **lambda functions**
-

🧪 Example: Parallel Execution

- Suppose we want to increment all elements in `[1, 2, 3, ..., 9]`
 - FP allows Spark to split this into parts and **run them simultaneously across nodes**
 - Code remains **unchanged**, thanks to **functional purity and immutability**
-

🔄 Spark & Functional Programming

- Spark is:
 - Written in Scala → naturally supports **functional programming**
 - **Parallel by design**: All FP-based Spark jobs run in distributed fashion
 - Ideal for **data transformations, analytics, and ML pipelines**
-

✿ Key Takeaways

- Functional Programming is a **declarative paradigm** centered on **what to compute**, not **how**
 - **Lambda functions** enable short, anonymous expressions in FP
 - Apache Spark uses **FP + lambda functions** to distribute and parallelize big data computations
 - **FP = Scalability + Simplicity + Parallelism**, especially in the context of Spark
-

■ lesson 3: Parallel Programming using Resilient Distributed Datasets

🧠 Objectives

- Define **Resilient Distributed Datasets (RDDs)**
 - Understand the concept of **parallel programming**
 - Explain how **resilience** works in Apache Spark
 - Relate **RDDs to parallel processing** in Spark
-

✂ Key Concepts / Steps

◆ What is an RDD?

An **RDD (Resilient Distributed Dataset)** is Spark's **primary data abstraction**. It is:

- A distributed collection of elements
- **Immutable** (once created, it cannot be changed)
- **Fault-tolerant**
- Partitioned across the cluster for **parallel operations**

◆ How to Create an RDD

You can create an RDD in three main ways:

1. **From an external or local file**, using sources like HDFS, S3, Cassandra, HBase, etc.
2. **Using the `parallelize()` function** on a local collection in the driver program.
3. **By transforming an existing RDD** into a new one (e.g. using `.map()`, `.filter()`).

You can also **manually define the number of partitions**, which affects parallelism (typically 2-4 partitions per CPU core is optimal).

◆ What is Parallel Programming?

Parallel programming means:

- Dividing a computational task into smaller parts
- Solving them **simultaneously** using multiple processors
- Accessing a **shared memory pool**
- Achieving efficiency via **coordination and task splitting**

◆ How RDDs Enable Parallelism

- RDDs are **inherently parallel**: they're divided into partitions
- Each partition is processed by a **separate task** on a node
- This enables high-performance distributed computation

◆ Resilience in RDDs

RDDs are **resilient** because:

- They are **immutable** and thus always **recoverable**
 - Spark can **persist/cache** RDDs in memory for **faster repeated access**
 - This caching mechanism is also **fault-tolerant**
 - Cached/persisted RDDs speed up **iterative and interactive computations** (up to 10x)
-

Example

```
# Python Example of creating an RDD from a list
data = [1, 2, 3, 4, 5]
rdd = sc.parallelize(data, 4) # 4 partitions
```

```
// Scala version
val data = List(1, 2, 3, 4, 5)
val rdd = sc.parallelize(data, 4)
```

Key Takeaways

- **RDDs** are the backbone of Spark's data abstraction — fault-tolerant, distributed, and immutable.
 - You can create RDDs from files, collections, or other RDDs.
 - **Parallel programming** is about breaking tasks down and solving them concurrently using multiple processors.
 - RDDs enable **parallelism by design**, thanks to their partitioned nature.
 - **Persistence** and **immutability** make RDDs **resilient** and suitable for iterative tasks in memory.
-

Lesson 4: Scale/Out and Data Parallelism in Apache Spark

Objectives

- Identify the main components of Apache Spark's architecture
 - Understand how Spark handles **parallelism** and **scaling** in big data contexts
 - Explain how Spark applications are structured and executed across a cluster
-

Key Concepts

◆ Apache Spark Architecture: 3 Core Components

1. **Data Storage**
 - Spark loads data from **Hadoop-compatible sources** (e.g., HDFS, S3, Cassandra).
 - Data flows from storage **into memory** for processing.
2. **High-Level Programming APIs**
 - Spark supports APIs in **Scala, Python (PySpark), and Java**.
 - These APIs enable users to define operations using **RDDs, DataFrames**, etc.
3. **Cluster Management Framework**
 - Responsible for **distributed resource management** and task scheduling.
 - Spark supports:
 - **Standalone cluster manager**
 - **Apache Mesos**
 - **YARN (Yet Another Resource Negotiator)**



Visualization:

Data → API/Compute Interface → Distributed Nodes (for parallel processing)

◆ Spark Core

- Known as the **base engine** behind what people call "Spark"
 - Handles:
 - **Memory management**
 - **Task scheduling**
 - **RDD API definitions**
 - **Parallelization of distributed collections**
 - **Fault-tolerant** by design — essential for big data reliability
-

◆ Spark Application Architecture

1. **Driver Program**
 - Acts as the "executive management"
 - Responsible for:
 - Defining Spark jobs
 - Splitting jobs into **tasks**
 - Assigning tasks to executors
 - Collecting results
 - Initiates communication with the cluster
2. **Executor Program**
 - Lives on **worker nodes**
 - Executes the tasks assigned by the driver
 - Can use **multiple cores** per executor
 - Supports **multithreaded processing**
3. **Worker Nodes**

- Physical/virtual machines running executors
- Can scale **horizontally** by adding more workers
- Think of them as "office buildings" in a company analogy

💡 Analogy:

- **Driver** = Executive team making decisions
 - **Executors** = Employees doing the work
 - **Workers** = Office buildings where employees work
-

🔧 Spark in Action

- **Scaling Spark:**
You can add more worker nodes incrementally to **scale out** processing power.
 - **Parallelism:**
Spark distributes RDDs and tasks across executors, enabling **parallel execution** and **distributed data processing**.
-

✖ Key Takeaways

- Apache Spark is composed of:
 1. **Data layer**
 2. **Compute APIs**
 3. **Cluster manager**
 - **Spark Core** is the heart of Spark: fault-tolerant, memory-aware, and optimized for **large-scale data**.
 - The **Driver-Executor-Worker** architecture provides **fine-grained control** over parallelism and task management.
 - Spark **scales linearly** with more worker nodes, making it ideal for **big data environments**.
-

📘 Lesson 5: SparkSQL and DataFrames

🧠 Learning Objectives

- Define **SparkSQL** and identify the parts of a **Spark SQL query**
 - Understand the concept of **DataFrames**, their query components, and their benefits
 - Explain how **DataFrames integrate** with SparkSQL
-

What is SparkSQL?

- **SparkSQL** is a Spark module for **structured data processing**.
 - You can interact with SparkSQL via:
 - **SQL queries**
 - **DataFrame API**
 - Supported languages: **Java, Scala, Python, R**
 - Regardless of the language/API used, SparkSQL runs on the **same execution engine**.
-

Example: Spark SQL Query (Python)

```
spark.sql("SELECT * FROM people")
```

- `people` must be registered as a **temporary view** beforehand.
-

SparkSQL Advantages Over RDDs

- **Cost-based query optimization**
 - **Columnar storage**
 - **Code generation** for better performance
 - Deeper insight into data structure for advanced optimizations
-

What is a DataFrame?

- A **DataFrame** is a collection of data **organized into named columns**
 - Similar to:
 - A **table** in a relational DB
 - A **data.frame** in R or **Pandas** in Python
 - Built on **top of RDD API**, enabling relational-style queries
-

Creating a DataFrame from JSON (Python)

```
df = spark.read.json("people.json")  
df.createOrReplaceTempView("people")
```

- This allows SQL queries to be run on the DataFrame
-

☀️ DataFrame Benefits

- **Highly scalable:** from small local files to petabytes of cluster data
 - Supports **many formats** and storage systems
 - Optimized via **Catalyst optimizer**
 - **Developer-friendly:** APIs for Python, Scala, Java, and R
-

💻 Query Examples

Show names column (3 methods, same result):

1. SQL: `SELECT name FROM people`
2. DataFrame API:

```
df.select("name")  
df.select(df.name)
```

Filter by age > 21:

1. SQL: `SELECT * FROM people WHERE age > 21`
2. DataFrame API:

```
df.filter(df.age > 21)
```

📄 Conclusion

- **SparkSQL:** a module for structured processing with SQL-like and programmatic access
 - **DataFrames:** powerful, optimized, table-like abstractions enabling distributed SQL queries
 - Together, they offer a flexible, performant approach for big data analytics
-

Module 3 Cheat Sheet: Apache Spark

🧠 Objectives

- Learn the **essential PySpark methods** and functions used in Apache Spark for data processing.
 - Understand **SparkSession**, **DataFrame operations**, **RDD transformations**, and **SQL integration** within PySpark.
 - Get familiar with common **commands**, **data caching**, and **schema inspection**.
-

⌘ Key Concepts / Steps

🔧 Spark Initialization

- `SparkSession.builder.appName("MyApp").getOrCreate()`
Initializes Spark and names your application.

💾 Caching & Persistence

- `df.cache()`
Caches a DataFrame/RDD in memory to optimize repeated operations.

📊 DataFrame Actions

- `df.count()` – Returns row count.
- `df.show()` – Displays tabular content.
- `df.printSchema()` – Displays schema as a tree.

🧠 View Creation & SQL

- `df.createOrReplaceTempView("cust_tbl")` – Creates temporary SQL-accessible view.
- `spark.sql("SELECT * FROM cust_tbl")` – Executes SQL queries on the view.

🔍 Data Filtering & Selection

- `df.filter(df['age'] > 30)` – Filters DataFrame by condition.
- `df.select('name', 'age')` – Selects specific columns.

🧱 RDD Operations

- `df.rdd.map(lambda row: (row['name'], row['age']))` – Transforms rows.
- `sc.parallelize([1,2,3])` – Creates RDD from a list.

📦 File Reading

- `spark.read.csv("file.csv")` – Reads CSV as DataFrame.
- `spark.read.json("file.json")` – Reads JSON files.

🧰 Miscellaneous

- `import` – Imports modules.
- `pip install pyspark` – Installs Spark in your environment.
- `from pyspark.sql.functions import current_timestamp` – Gets current time in Spark.

Examples

```
# Starting Spark
from pyspark.sql import SparkSession
spark = SparkSession.builder.appName("MyApp").getOrCreate()

# Reading and Caching Data
df = spark.read.csv("customer.csv")
df.cache()

# Filtering & Showing Data
filtered_df = df.filter(df['age'] > 30)
filtered_df.show()

# SQL Example
df.createOrReplaceTempView("cust_tbl")
result = spark.sql("SELECT name, age FROM cust_tbl WHERE age > 30")
result.show()

# Get Current Timestamp
from pyspark.sql.functions import current_timestamp
df.select(current_timestamp().alias("current_time")).show()
```

Key Takeaways

- **SparkSession** is your entry point to any Spark functionality.
 - **DataFrame APIs** like `.filter()`, `.select()`, and `.show()` allow easy, expressive data manipulations.
 - Use `.cache()` wisely when running **repeated actions** on the same data.
 - SQL integration in Spark makes it easier to query structured data on the fly.
 - Spark can read data from multiple formats: **CSV, JSON**, etc.
 - Mastering these functions is foundational for working on **big data pipelines** and **ETL processes**.
-

Apache Spark Crash Recap

(Your 3-in-1 Summary: Concepts + Cheat Sheet + Glossary)

Core Concepts

1. What is Spark?

Apache Spark is a **fast, in-memory distributed processing engine** used for big data workloads. It allows processing large datasets **in parallel across multiple nodes**.

2. Main Components of Spark:

Component	Purpose
Spark Core	The base engine for memory management, task scheduling, fault tolerance
Spark SQL	For structured data and SQL queries
Spark Streaming	For real-time stream processing
MLlib	Built-in Machine Learning library
GraphX	For graph computation

3. Spark Architecture Basics:

Role	Description
Driver	The boss! It plans, coordinates, and monitors execution
Executor	The workers. They run computations on the data
Cluster Manager	Assigns resources (e.g. Spark Standalone, YARN, Kubernetes)
Worker Node	The machines that host Executors

 *Driver gives tasks to Executors, they do the work, and report back. Classic delegation!*

🔗 Handy Cheat Sheet (🔥 Most-Used Spark Commands)

Command	What it does
<code>spark.read.csv(...)</code>	Load data from a CSV file
<code>df.printSchema()</code>	Show data structure
<code>df.select("column")</code>	Select specific columns
<code>df.filter(df.age > 30)</code>	Filter rows using condition
<code>df.groupBy(...).agg(...)</code>	Perform aggregations
<code>df.createOrReplaceTempView("temp")</code>	Create temporary table for SQL
<code>spark.sql("SELECT * FROM temp")</code>	Run SQL on the table
<code>df.cache()</code>	Cache results to memory
<code>df.count()</code>	Count rows (also triggers transformations)

🧩 Key Glossary Terms

Term	Meaning
RDD	Resilient Distributed Dataset (core data structure in Spark)
DataFrame	Tabular data structure, similar to Pandas (built on top of RDDs)
In-memory	Spark processes data in RAM, not from disk → much faster
Lazy Evaluation	Spark only computes when an action like <code>.count()</code> is called
Transformations	Operations like <code>filter()</code> , <code>select()</code> → return a new DataFrame
Actions	Operations like <code>count()</code> , <code>collect()</code> → trigger execution
Fault Tolerance	Automatic recovery from failures using lineage and DAG tracking
Parallelism	Distributing tasks across cores/machines for faster performance

🧪 Example Flow

```
# Read CSV
df = spark.read.csv("employees.csv", header=True, inferSchema=True)

# Filter rows
df_filtered = df.filter(df.salary > 100000)

# Select columns
df_selected = df_filtered.select("name", "department")

# Create SQL view and run query
df_selected.createOrReplaceTempView("high_paid")
spark.sql("SELECT * FROM high_paid WHERE department = 'Data'").show()
```

🧠 Quick Memory Hacks

- **RDD** → Think of it as the raw data block: "Really Distributed Donuts 🍩"
 - **Driver vs Executor** → The **driver is the manager**, the **executors are the employees**.
 - **Lazy Evaluation** → Spark's motto: *"Why work unless I have to?"*
-

✅ Key Takeaways

- Spark is **scalable**, **fast**, and supports multiple languages (Python, Scala, Java).
 - You can interact with Spark using **DataFrames**, **SQL**, or **RDDs**.
 - **Transformations are lazy**, actions trigger computations.
 - For performance: **cache frequently used data**.
 - Spark fits many use cases: ETL, streaming, ML, and interactive queries.
-

Lesson's Summary:

- Spark is an open-source, in-memory application framework for distributed data processing and iterative analysis on massive data volumes. Both distributed systems and Apache Spark are inherently scalable and fault-tolerant.
 - Apache Spark solves the problems encountered with MapReduce by keeping a substantial portion of the data required in memory, avoiding expensive and time-consuming disk I/O.
 - Functional programming follows a declarative programming model that emphasizes *what* instead of *how-to* and uses expressions.
 - Lambda functions or operators are anonymous functions that enable functional programming. Spark parallelizes computations using lambda calculus, and all functional Spark programs are inherently parallel.
 - Resilient distributed datasets, or RDDs, are Spark's primary data abstraction consisting of a fault-tolerant collection of elements partitioned across the cluster's nodes, capable of accepting parallel operations.
 - You can create an RDD using an external or local Hadoop-supported file, from a collection, or from another RDD. RDDs are immutable and always recoverable, providing resilience in Apache Spark. RDDs can persist or cache datasets in memory across operations, which speeds up iterative operations in Spark.
 - Apache Spark architecture consists of components data, compute input, and management. The fault-tolerant Spark Core base engine performs large-scale big data-worthy parallel and distributed data processing jobs, manages memory, schedules tasks, and houses APIs that define RDDs.
 - Spark SQL provides a programming abstraction called DataFrames and can also act as a distributed SQL query engine.
 - Spark DataFrames are conceptually equivalent to a table in a relational database or a data frame in R or Python, but with richer optimizations.
-

Module Four

DataFrames And Spark SQL

Introduction to DataFrames and Spark SQL

■ Lesson 1: RDDs in Parallel Programming and Spark

Objectives

- Describe what Resilient Distributed Datasets (RDDs) are
 - Explain how RDDs are used in Spark
 - Differentiate between RDD transformations and actions
 - List key transformations and actions used in Spark
-

✂ Key Concepts / Steps

◆ Resilient Distributed Datasets (RDDs)

- RDDs are **Spark's core data abstraction**, representing a distributed collection of objects across a cluster
- They are **partitioned** and processed in parallel

◆ Transformations

- **Transformations** create new RDDs from existing ones
- They are **lazy**, meaning they don't compute results until an action is called
- Common transformations:
 - `map`: applies a function to each element
 - `filter`: selects elements that match a condition
 - `flatMap`: similar to `map` but returns multiple output elements for each input
 - `distinct`: removes duplicate elements

◆ Actions

- **Actions** trigger the actual computation and return values to the **driver program**
- Common actions:
 - `reduce`: aggregates elements using a function
 - `collect`: retrieves the entire RDD to the driver
 - `take(n)`: retrieves the first `n` elements
 - `takeOrdered(n)`: retrieves `n` smallest elements in order

◆ DAG (Directed Acyclic Graph)

- Spark builds a **DAG** to schedule operations on RDDs
- Nodes = RDDs, Edges = Transformations/Actions
- DAG enables **fault tolerance**: if a node fails, Spark can **reconstruct** the RDD from lineage

◆ How Execution Works

1. Spark creates a DAG when you define an RDD
2. When an action is called, Spark **schedules** the tasks via the DAG scheduler
3. The scheduler computes transformations and sends the result to the driver

Examples

Example 1: Map + Collect

```
def f(x): return x - 1
rdd = sc.parallelize([1, 2, 3])
result = rdd.map(f).collect()
# Output: [0, 1, 2]
```

- `map(f)`: transforms each element using `f`
 - `collect()`: gathers the result into the driver
-

Key Takeaways

- **RDDs** are the foundation of Spark's data processing—fault-tolerant, distributed, and efficient
 - **Transformations** are lazy and define *what* should be done
 - **Actions** trigger the computation and define *when* it should be done
 - The **DAG** structure ensures operations are well-ordered and fault-tolerant
 - Mastering RDD transformations and actions is critical for effective Spark development
-

Lesson 2: Data-frames and Datasets

Objectives

- Describe the features and benefits of **Datasets** in Apache Spark.
 - List **three ways** to create datasets in Spark.
 - Compare and contrast **Datasets vs DataFrames**.
-

✂ Key Concepts / Steps

📘 What is a Dataset?

- A **Dataset** is a **strongly-typed**, distributed collection of **JVM objects**.
- Combines features of **RDDs** (e.g., lambda functions, type safety) and **DataFrames** (e.g., SQL optimizations).
- Only supported in **Scala** and **Java**, due to their **static typing**.
- Not available in **Python** or **R**, which are dynamically typed.

💡 Key Features of Datasets:

- **Immutable** – cannot be changed once created.
- Use an **Encoder** to convert JVM objects into a tabular format.
- Extend the **DataFrame API** – essentially a DataFrame of typed Rows.
- Enable **compile-time type safety** – errors are caught before execution.
- **Faster execution** than RDDs for aggregates, thanks to **Catalyst** and **Tungsten** optimizations.
- More efficient **memory usage and caching** due to Spark's understanding of the data structure.
- Include convenient **high-level operations** like `.sum`, `.avg`, `.groupBy`, and `.join`.

🔧 Examples

1. From a Scala sequence

```
val mySeq = Seq(1, 2, 3)
val myDataset = mySeq.toDS()
```

2. From a text file with an explicit schema

```
val textDS = spark.read.textFile("file.txt").as[String]
```

3. From a JSON file using a custom class

```
case class Customer(name: String, id: Int)
val customerDS = spark.read.json("customers.json").as[Customer]
```

✂ Key Takeaways

- **Datasets** combine the best of **RDDs** and **DataFrames**.
- They are **strongly typed**, **optimized**, and **efficient**.
- Only usable in **Java** and **Scala** due to static typing requirements.
- More **type-safe** and **faster** than RDDs, and offer **SQL-style operations** like DataFrames.

- **Datasets extend DataFrames**, which themselves extend RDDs — think of them as Spark’s most powerful abstraction (for typed data).
-

Lesson 3: Catalyst and Tungsten

Objectives

By the end of this lesson, you should be able to:

- Understand the **goals** of Spark SQL optimization.
 - Describe the **Catalyst** and **Tungsten** optimizers in Spark.
 - Explain how **Spark SQL** achieves both **query** and **memory optimization**.
-

✂ Key Concepts / Steps

✿ Spark SQL Optimization Goals

- **Improve query performance**: Reduce execution time and memory consumption.
- **Save time and cost** for organizations running large-scale data queries.

Catalyst Optimizer (Rule-Based & Cost-Based)

- **Built-in rule-based optimizer** in Spark SQL.
- Based on **Scala functional programming**.
- **Extensible**: Developers can add custom rules or support new data types.

Catalyst Optimization Phases:

1. **Analysis** – Analyzes the query and generates an *unresolved logical plan* using metadata (catalog).
2. **Logical Optimization** – Applies rule-based transformations like constant folding, predicate pushdown, column pruning.
3. **Physical Planning** – Generates *multiple physical plans* and selects the best using a *cost model*.
4. **Code Generation** – Compiles the chosen physical plan into *Java bytecode* for distributed execution.

Analogy:

Rule-based optimization is like tuning a car with the right tires and oil.

Cost-based optimization is like planning the fastest route for a road trip.

⚙️ Tungsten (Hardware-Level Optimization)

- Spark's engine for **low-level memory & CPU optimization**.
- Addresses limitations of the **JVM** for big data processing.

Key Features of Tungsten:

- **Manual memory management:** Bypasses JVM's garbage collector.
 - **Cache-friendly data structures** using STRIDE access (avoiding random access).
 - **JVM bytecode generation on demand** for efficient execution.
 - **Avoids virtual function dispatch:** Minimizes CPU overhead.
 - **Places data in CPU registers** and uses **loop unrolling** to improve performance.
-

🔧 Examples

While no explicit code was shown, the analogy of tuning and routing a car was used to illustrate:

- **Rule-based optimization** = engine tuning
 - **Cost-based optimization** = choosing the fastest travel path
-

⚙️ Key Takeaways

- **Catalyst** handles SQL **query optimization** with a four-phase process: analysis, logical optimization, physical planning, and code generation.
 - **Tungsten** focuses on **low-level memory and CPU optimization**, enhancing performance by working closer to the hardware layer.
 - Together, **Catalyst + Tungsten** make Spark SQL powerful for running **fast, efficient, and scalable big data queries**.
-

📘 Lesson 4: User-Defined Schema (UDS) for DSL and SQL

🧠 Objectives

- Understand how to **define and apply a user-defined schema (UDS)** in PySpark.
 - Learn how to **read structured data** (like CSV files) with **explicit data types**.
 - Validate the schema using `printSchema()`.
-

✂ Key Concepts / Steps

1. Import Required Libraries

To define schemas manually, we use:

```
from pyspark.sql.types import StructType, IntegerType, FloatType,
StringType, StructField
```

2. Define the Schema

Before defining, explore the data to understand its structure and datatypes.
Then define it using `StructType` and `StructField`.

Example schema:

```
schema = StructType([
    StructField("Emp_Id", StringType(), False),
    StructField("Emp_Name", StringType(), False),
    StructField("Department", StringType(), False),
    StructField("Salary", IntegerType(), False),
    StructField("Phone", IntegerType(), True),
])
```

- `False` → field cannot be null
- `True` → field can be null

3. Read the CSV File Using the Schema

Apply the schema when reading the file:

```
df = (spark.read
    .format("csv")
    .schema(schema)
    .option("header", "true")
    .load("employee.csv")
)
df.show()
```

4. Validate Schema with `printSchema()`

```
df.printSchema()
```

Confirms that your schema has been properly applied to the DataFrame.

Example

CSV File Content:

```
emp_id,emp_name,dept,salary,phone
A101,jhon,computer science,1000,+1 (701) 846 958
A102,Peter,Electronics,2000,
A103,Micheal,IT,2500,
```

Schema Declaration:

```
StructField("Emp_Id", StringType(), False),
StructField("Emp_Name", StringType(), False),
StructField("Department", StringType(), False),
StructField("Salary", IntegerType(), False),
StructField("Phone", IntegerType(), True),
```

⚠ This schema will **fail on the Phone field** due to invalid formatting (+1 (701) ... is not an Integer). You'd either need to:

- Use `StringType()` for Phone, or
 - Pre-process the CSV to clean phone numbers.
-

🌱 Key Takeaways

- **User-Defined Schemas** help ensure **data consistency** by explicitly setting data types and nullability.
 - Use `StructType` and `StructField` to define the schema.
 - Always explore your data **before schema definition** to avoid mismatches.
 - Use `printSchema()` to confirm the applied schema.
 - Be mindful of data types—real-world data is often messy and may require cleaning or flexible types (e.g., use `StringType()` for mixed-format columns).
-

📘 Lesson 5: ETL with DataFrames

🎯 Learning Objectives

- Identify the basic DataFrame operation steps
 - Apply basic Spark DataFrame operations on real-world data
 - Use Spark DataFrames in an end-to-end data processing pipeline
-

🧰 Basic DataFrame Operation Steps

The lifecycle of a Spark DataFrame in an ETL pipeline consists of:

1. **Reading**
 - Load data into a DataFrame (from CSV, JSON, DB, etc.)
 - Can be read directly or converted from a Pandas DataFrame
2. **Analysis**
 - View schema: `printSchema()`
 - Show rows: `show(n)`

- Inspect columns: `select("column_name")`
- Get data types, stats, and trends
- 3. **Transformation**
 - **Filter:** e.g., `filter(mpg < 18)`
 - **Sort:** e.g., `orderBy("column")`
 - **Join** with other DataFrames
 - **Column operations:** conversions, calculations
 - **Aggregation:** group by, sum, average, etc.
- 4. **Loading**
 - Load transformed data into a new database or file system
- 5. **Writing**
 - Write to disk (e.g., .json, .parquet)
 - Export to a database (e.g., PostgreSQL via JDBC or API)

ETL vs. ELT

ETL (Extract → Transform → Load) ELT (Extract → Load → Transform)

Used with **Data Warehouses**

Used with **Data Lakes**

Transforms happen **before loading**

Transforms happen **after loading**

Predefined schema/usage

Flexible, schema-on-read

Better for **structured data**

Ideal for **big/unstructured data**

Organizations often mix ETL and ELT depending on use case.

Real-World Example: *mtcars* Dataset

Columns include:

- mpg (miles per gallon)
- cyl (number of cylinders)
- hp (horsepower)
- And other car performance metrics

Analysis Examples:

```
df.printSchema()
df.show(5)
df.select("mpg").show(5)
```

Transformation Examples:

```
df.filter(df["mpg"] < 18)
df.groupBy("cyl").count().orderBy("count")
```


Writing the Final Output

After transformations:

- Save as .json, .csv, .parquet, etc.
 - Write to SQL databases using APIs or connectors
-

Key Takeaways

- DataFrames support an efficient, readable way to handle real-world data.
 - **ETL** and **ELT** strategies depend on data architecture (warehouse vs. lake).
 - Analysis and transformation steps are flexible and domain-dependent.
 - Spark integrates with Pandas for easy prototyping and conversion.
-

Lesson 6: Apache Spark SQL; Real-world usage of SparkSQL

Objectives

- Define what **Spark SQL** is
 - Create **temporary** and **global temporary views** in Spark SQL
 - Perform **data aggregation** using SQL queries
 - Recognize the **data sources** Spark SQL supports
-

Key Concepts / Steps

◆ What is Spark SQL?

- A **module in Apache Spark** for structured data processing
 - Allows running **SQL queries** on **Spark DataFrames**
 - APIs available in **Java, Scala, Python, and R**
-

◆ Views in Spark SQL

Temporary View:

- **Local scope** (exists in current Spark session only)
- Good for isolated session queries

```
df.createOrReplaceTempView("people")
```

```
spark.sql("SELECT * FROM people").show()
```

Global Temporary View:

- **Global scope** (shared across Spark sessions)
- Must be accessed via the `global_temp` database

```
df.createOrReplaceGlobalTempView("people")
spark.sql("SELECT * FROM global_temp.people").show()
```

◆ Aggregation with Spark SQL

Two approaches to aggregate data:

1. **Using DataFrame functions**
(e.g., `groupBy()`, `agg()`, `count()`, `avg()`)
 2. **Using SQL queries on temp views**
 - Create a temp view
 - Run SQL `GROUP BY` query
 - Example:
 3. `SELECT cylinders, COUNT(*)`
 4. `FROM cars`
 5. `GROUP BY cylinders`
 6. `ORDER BY cylinders DESC`
-

◆ Supported Data Sources

Spark SQL can read and write to multiple sources:

- **Parquet** (columnar format, schema preserved)
 - **JSON** (schema inferred)
 - **Hive tables** (for those using Hadoop ecosystem)
-

Examples

```
# Example 1: Creating Temp View
df = spark.read.json("people.json")
df.createOrReplaceTempView("people")
spark.sql("SELECT name, age FROM people").show()

# Example 2: Global Temp View
df.createOrReplaceGlobalTempView("people")
spark.sql("SELECT * FROM global_temp.people").show()

# Example 3: Aggregation using SQL
df.createOrReplaceTempView("cars")
spark.sql("""
    SELECT cylinders, COUNT(*)
```

```
FROM cars
GROUP BY cylinders
ORDER BY cylinders DESC
""").show()
```

✿ Key Takeaways

- Spark SQL bridges the world of **DataFrames** and **SQL-style queries**
 - Views are essential for SQL:
 - Use **Temporary Views** for one session
 - Use **Global Temporary Views** for sharing across sessions
 - Aggregation can be done via **DataFrame methods** or **SQL**
 - Spark SQL supports **Parquet**, **JSON**, and **Hive**, giving flexibility for data sources
-

📖 Lesson 7: Common Transformations and Optimization Techniques in Spark

🧠 Objectives

- Distinguish between **narrow and wide transformations** in Spark
- Apply **rule-based** and **cost-based** optimization techniques
- Use **PySpark transformations** effectively for performance tuning
- Understand how Spark minimizes shuffling, memory usage, and CPU overhead

🔧 Key Concepts / Steps

◆ Narrow vs. Wide Transformations

Transformation Type	Description	Impact
Narrow	Operate within a single partition	✅ Fast, ❌ No shuffle
Wide	Require data across partitions	❌ Slow, ✅ Shuffle-heavy

📦 Examples of Narrow Transformations:

- `map()` – Apply a function to each item
- `filter()` – Keep only items that match a condition
- `union()` – Combine two datasets

```
rdd = sc.parallelize([1, 2, 3, 4, 5])
mapped = rdd.map(lambda x: x * 2)           # [2, 4, 6, 8, 10]
filtered = rdd.filter(lambda x: x % 2 == 0)  # [2, 4]
```

✿ Examples of Wide Transformations:

- `groupBy()` – Group by key, then aggregate
- `join()` – Merge two RDDs by key
- `sortBy()` – Sort data, potentially across partitions

```
rdd1 = sc.parallelize([("apple", 2), ("banana", 3)])
rdd2 = sc.parallelize([("apple", 5), ("banana", 1)])
joined = rdd1.join(rdd2) # [('apple', (2, 5)), ('banana', (3, 1))]
```

◆ Rule-Based Optimization Techniques (DataFrame API)

Technique	What It Does
Predicate Pushdown	Filters data close to the source to reduce volume early
Constant Folding	Evaluates constant expressions at compile-time
Column Pruning	Removes unused columns from the query plan
Join Reordering	Changes the join order to reduce intermediate data size

```
df1.filter(col("age") > 25) # Predicate Pushdown
df1.select(col("name"), col("age") + 2) # Constant Folding
df1.select(col("name")) # Column Pruning
df1.join(df2, on="name") # Join Reordering
```

◆ Cost-Based Optimization (CBO) in Spark

Feature	Description
Adaptive Query Execution	Spark adjusts execution plan on-the-fly
Cost-Based Join Reordering	Spark uses stats to pick the best join strategy
Broadcast Hash Join	Small table is broadcast to all nodes to avoid shuffle
Shuffle Partitioning	Optimizes how data is split and shuffled during groupBy/aggregations

```
spark.conf.set("spark.sql.adaptive.enabled", "true") # Enable AQE
df1.join(df2, on="name") # Join auto-optimized based on runtime stats
```

🧪 Examples

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col

spark = SparkSession.builder.appName("Optimizations").getOrCreate()
```

```
# Sample Data
df1 = spark.createDataFrame([("Alice", 25), ("Bob", 30)], ["name",
"age"])
df2 = spark.createDataFrame([("Alice", "NY"), ("Bob", "SF")],
["name", "city"])

# Enable Adaptive Query Execution
spark.conf.set("spark.sql.adaptive.enabled", "true")

# Rule-based optimizations
filtered_df = df1.filter(col("age") > 25)           # Predicate Pushdown
folded_df = filtered_df.select(col("age") + 2)      # Constant Folding
pruned_df = folded_df.selectExpr("` (age + 2) ` as age_plus_2") #
Column Pruning

# Cost-based optimized join
optimized_join = df1.join(df2, on="name")          # Adaptive join plan

optimized_join.show()
spark.stop()
```

Key Takeaways

- **Narrow transformations** are fast and efficient – think local, no data movement
 - **Wide transformations** are powerful but expensive – involve shuffling and coordination
 - **Rule-based optimizations** (like predicate pushdown) simplify and speed up query plans
 - **Cost-based optimizations** (like adaptive execution) let Spark auto-tune queries during runtime
 - Using the right **transformation** and enabling **optimization flags** can **significantly** reduce compute time and memory use
-

Lesson's Summary:

- RDDs are Spark's primary data abstraction partitioned across the cluster's nodes.
- Spark uses directed acyclic graphs (DAGs) to enable fault tolerance. When a node goes down, Spark replicates the DAG and restores the node.
- Transformations undergo lazy evaluation, meaning they are only evaluated when the driver function calls an action.
- A data set is a distributed collection of data that provides the combined benefits of both RDDs and SparkSQL.

- Consisting of strongly typed JVM objects, data sets use DataFrame typesafe capabilities and extend object-oriented API capabilities.
 - Data sets work with both Scala and Java APIs. DataFrames are not typesafe. You can use APIs in Java, Scala, and Python. Data sets are Spark's latest data abstraction.
 - Spark SQL optimization's primary goal is to improve a SQL query's run-time performance by reducing the query's time and memory consumption, saving organizations time and money.
 - Catalyst is the Spark SQL built-in rule-based *query* optimizer. Catalyst performs analysis, logical optimization, physical planning, and code generation.
 - Tungsten is the Spark built-in cost-based optimizer for CPU and memory usage that enables cache-friendly computation of algorithms and data structures.
 - Basic DataFrame operations are reading, analysis, transformation, loading, and writing.
 - You can use a Pandas DataFrame in Python to load a data set and apply the `'printschema'`, `'select'`, or `'show'` function for data analysis.
 - Keep only relevant data for transform tasks and apply functions such as filters, joins, column operations, grouping and aggregations, and other functions.
 - Spark SQL consists of Spark modules for structured data processing that can run SQL queries on Spark DataFrames and are usable in Java, Scala, Python, and R.
 - Spark SQL supports both temporary views and global temporary views. Use a DataFrame function or an SQL Query and Table view for data aggregation. Spark SQL supports Parquet files, JSON data sets, and Hive tables.
-
-

Module 4 Cheat Sheet: Spark SQL & DataFrames

Spark Initialization & Session Setup

Method	Description	Code Example
<code>SparkSession.builder.appName()</code>	Sets app name shown in Spark UI	<pre>spark = SparkSession.builder.appName("MyApp").getOrCreate()</pre>
<code>SparkSession.builder.getOrCreate()</code>	Gets existing or creates a new session	<pre>spark = SparkSession.builder.getOrCreate()</pre>
<code>SparkContext()</code>	Creates a SparkContext (RDD-level)	<pre>sc = SparkContext("local", "MyApp")</pre>

Creating and Loading Data

Method	Description	Code Example
<code>createDataFrame()</code>	Creates a DataFrame from data	<pre>df = spark.createDataFrame(data, columns)</pre>
<code>spark.read.json()</code>	Loads JSON into DataFrame	<pre>json_df = spark.read.json("file.json")</pre>
<code>pd.read_csv()</code>	Loads CSV into pandas DataFrame	<pre>df = pd.read_csv("file.csv")</pre>

Data Exploration & Display

Method	Description	Code Example
<code>show()</code>	Displays DataFrame (default: 20 rows)	<pre>df.show()</pre>
<code>printSchema()</code>	Displays schema of DataFrame	<pre>df.printSchema()</pre>
<code>head(n)</code>	Returns first n rows	<pre>df.head(5)</pre>
<code>select()</code>	Select specific columns	<pre>df.select('name', 'age')</pre>

Filtering & Sorting

Method	Description	Code Example
<code>filter()</code> / <code>where()</code>	Filters rows	<code>df.filter(df['age'] > 30)</code> / <code>df.where(df['age'] > 30)</code>
<code>sort()</code>	Sorts rows by column(s)	<code>df.sort("age")</code> or <code>df.sort(["age", "name"], ascending=[False, True])</code>

Aggregation & Grouping

Method	Description	Code Example
<code>groupBy().agg()</code>	Group and apply aggregation	<code>df.groupBy("age").agg({"age": "count"})</code>

Working with Views and SQL

Method	Description	Code Example
<code>createOrReplaceTempView()</code>	Creates a temporary SQL view	<code>df.createOrReplaceTempView("cust_tbl")</code>
<code>spark.sql()</code>	Run SQL query on view	<code>result = spark.sql("SELECT * FROM cust_tbl WHERE age > 30")</code>

Data Cleaning & Column Operations

Method	Description	Code Example
<code>fillna()</code>	Replaces nulls	<code>df.fillna(0)</code>
<code>withColumn()</code>	Adds/modifies column	<code>df.withColumn("age_squared", col("age") ** 2)</code>
<code>withColumnRenamed()</code>	Renames a column	<code>df.withColumnRenamed("age", "years_old")</code>

UDFs (User-Defined Functions)

Method	Description	Code Example
<code>spark.udf.register()</code>	Register a custom function	
<pre>from pyspark.sql.functions import udf from pyspark.sql.types import StringType</pre>		


```
def my_udf(val): return val.upper()
spark.udf.register("my_udf", my_udf, StringType())
```

⚙️ Extras

```
| Command | Description | Example |
|-----|-----|-----|
| `pip install` | Install Python packages | `pip install pyspark` |
| `import` | Import necessary modules | `from pyspark.sql import SparkSession` |
```

💡 Pro Tip: Use DataFrames API for performance, but keep SQL queries handy for readable and familiar querying, especially when dealing with business logic or analysts.

🔍 Module 4 Terminology Summary: SparkSQL & DataFrames

💡 Core Concepts

Term	Quick Definition
Apache Spark	Open-source big data processing engine for distributed in-memory computation.
DataFrame	A distributed collection of data organized into columns (like a SQL table).
RDD (Resilient Distributed Dataset)	Low-level fault-tolerant collection of elements distributed across nodes.
Dataset	Type-safe version of DataFrame used mainly in Scala/Java.
SparkSQL	A Spark module that lets you run SQL queries on structured data.
Catalyst Optimizer	Spark’s query optimizer that transforms SQL code into efficient execution plans.
Tungsten Engine	Spark’s memory and CPU optimization backend for faster execution.

🔧 Transformations & Actions

Operation	Purpose
<code>filter(func)</code>	Filters rows based on a condition.
<code>map(func)</code>	Applies a function to each row.
<code>flatMap(func)</code>	Similar to <code>map</code> , but can return multiple elements per input row.
<code>distinct()</code>	Removes duplicate rows.
<code>groupBy().agg()</code>	Aggregates data by a key using functions like <code>count</code> , <code>avg</code> , <code>sum</code> , etc.

Operation	Purpose
<code>printSchema()</code>	Displays the structure (schema) of a DataFrame.

SQL with Spark

Term	Meaning
Temporary View	A session-scoped SQL view created from a DataFrame.
Global Temporary View	A persistent view across all sessions, accessed via <code>global_temp</code> database.
<code>createOrReplaceTempView()</code>	Creates or replaces a temp view from a DataFrame.
<code>spark.sql("...")</code>	Executes an SQL query within Spark.

Catalyst Optimizer Stages

1. **Analysis:** Converts SQL into a logical plan using a catalog.
 2. **Logical Optimization:** Applies rules like predicate pushdown and constant folding.
 3. **Physical Planning:** Selects optimal execution strategy.
 4. **Code Generation:** Converts plan into JVM bytecode for execution.
-

Data Formats

Format	Description
JSON	Lightweight, text-based data format. Spark can read/write JSON easily.
Parquet	Columnar storage format optimized for performance and space.
Hive Tables	Integration with Apache Hive data warehouses.

ETL vs ELT

Term	Definition
ETL	Extract → Transform → Load (classic pipeline model).
ELT	Extract → Load → Transform (used with Data Lakes, like in Spark).

Data Operations

Term	Explanation
Load / Export	Reading from or writing to external sources like JSON, JDBC, etc.

Term	Explanation
Schema	Structure of a DataFrame – column names and types.

Advanced Tech Terms

Term	What It Is
DAGScheduler	Manages task scheduling using DAGs (Directed Acyclic Graphs).
Driver Program	The main Spark process that manages coordination.
JVM (Java Virtual Machine)	Runtime environment for executing Spark code (written in Scala/Java).
Syntax Error	An error in code structure that prevents execution.

Module Five

Development and Runtime Environment Options

Spark Architecture

Lesson 1: Apache Spark Architecture

Objectives

- Describe the **Apache Spark architecture**
 - Identify the **Driver** and **Executor** roles in a Spark Application
 - Explain the relationship between **Jobs**, **Tasks**, and **Stages**
 - Define when a **Data Shuffle** occurs and why it's important
 - Understand **Client vs Cluster deploy modes** for the Driver
-

⚡ Key Concepts / Steps

1. Spark Application Components

- **Driver Process:** Runs the user code, defines the Spark Context, creates Jobs from user actions, and sends work to the cluster.
- **Executor Processes:** Run on worker nodes; execute tasks in parallel using threads and manage memory & data caching.

2. Spark Context

- Created in the Driver when the application starts.
- Needed before creating RDDs or DataFrames.
- Lives throughout the life of those datasets.

3. Jobs, Tasks, and Partitions

- **Job:** A parallelizable computation (e.g. a transformation triggered by an action like `.collect()`).
- **Task:** Smallest unit of execution — operates on a single **partition**.
- Jobs are divided into multiple tasks → tasks execute in **parallel** on executors.

4. Executors and Parallelism

- Each executor uses a portion of CPU & memory.
- Executors run **one task per core**.
- More executors = higher parallelism.

5. Stages and Shuffles

- A **Stage** = set of tasks on data that don't require shuffling.
- When a task needs data from another partition → **Data Shuffle** occurs.
- Shuffle is costly: requires **serialization, disk I/O, and network transfer**.
- Example: `groupByKey()` causes a shuffle between stages.

6. Driver Deploy Modes

- **Client Mode:** Driver runs on your machine (outside the cluster).
 - **Cluster Mode:** Driver is launched on a node **inside the cluster**.
 - In both modes, the driver must be able to **communicate with the cluster**.
-

Examples

◆ Map followed by GroupBy Example:

- Stage 1: `map` transformation on dataset “a” → creates dataset “b” with two partitions (1a, 2b).
 - Stage 2: `groupBy` causes a **shuffle**. Records are regrouped based on key, creating new partitions.
 - Final action (e.g. `collect`) sends data back to the driver.
-

Key Takeaways

- Spark has a **Driver** (brain) and many **Executors** (workers).
- **Jobs** are split into **Tasks**, and tasks operate on **Partitions**.
- A **Stage** = group of tasks that don't require reshuffling data.
- A **Shuffle** = data movement between partitions; separates stages and slows performance.
- Use **Client Mode** for local driver execution, **Cluster Mode** to run the driver within the cluster.

- **Avoid using** `.collect()` on large datasets — can overwhelm the driver's memory.

Lesson 2: Overview of Apache Spark Cluster Modes

Objectives

- Identify the different modes for running Apache Spark on a cluster
- Describe the components and benefits of each cluster mode
- Explain how to connect Spark with each mode
- Understand when and how to use **local mode** for Spark

Key Concepts / Steps

Cluster Managers – General Role

- A **cluster manager** communicates with a cluster to acquire resources for Spark applications
- It runs as a separate service and **abstracts the underlying cluster infrastructure**
- The **Spark Context** requests resources, and the cluster manager assigns them to **executors**, which then run the actual **tasks**

Cluster Modes Supported by Spark

1. Standalone Mode

- **Included by default** with Spark (no extra installation needed)
- Easiest and fastest to set up a Spark cluster
- Components:
 - **Master:** coordinates and tracks **workers**
 - **Workers:** run executors on cluster nodes
- Must avoid using all resources on worker if Master and Worker are on the same node
- Connect via Master's URL (e.g., `spark://hostname:port`)

2. Apache Hadoop YARN (Yet Another Resource Negotiator)

- **Widely used** in big data environments
- Spark connects using `--master yarn`
- Uses **Hadoop config files** for setup
- More complex than Standalone, but integrates well with other Hadoop tools

3. Apache Mesos

- A **general-purpose** cluster manager
- Allows dynamic resource sharing between Spark and other frameworks
- Supports **multiple Spark apps** with flexible partitioning
- May require extra configuration depending on setup

4. Kubernetes

- Manages **containerized** applications (Dockerized Spark jobs!)
 - Offers **portability, scalability, and easier dependency management**
 - Spark has **built-in Kubernetes support**
 - Connect using `--master k8s://<cluster-url>`
-

Local Mode

- Runs **Spark locally** as a single process
 - Executors run as **threads** in the main process
 - Great for **testing, debugging**, or running small jobs
 - Use: `--master local[n]`, where `n` = number of cores (or `local[*]` for all cores)
 - Not meant for performance or production
-

Key Takeaways

- Cluster managers allocate resources for Spark jobs and abstract infrastructure details
 - Spark supports multiple cluster modes: **Standalone, YARN, Mesos, and Kubernetes**, each with different setup needs and advantages
 - **Standalone** is easiest to get running
 - **YARN** is common in Hadoop environments
 - **Mesos** is flexible and scalable
 - **Kubernetes** is ideal for containerized, portable Spark apps
 - **Local mode** is great for development/testing but not scalable
-

📘 Lesson 3: How to Run an Apache Spark Application

🧠 Objectives

- Submit an Apache Spark application using `spark-submit`
 - Explain the unified interface nature of `spark-submit`
 - Use various configuration options to control application submission
 - Manage external dependencies effectively
 - Understand the benefits of using Spark Shell
-

⚡ Key Concepts / Steps

- **spark-submit Script:**
 - Found in the `bin/` directory
 - Unified interface to run apps on local or cluster modes (just change the `--master` argument)
 - Works with multiple languages (Java, Scala, Python)
 - **Basic Workflow:**
 1. Parses command-line options
 2. Reads configs from `spark-defaults`
 3. Uses `--master` to decide local vs. cluster
 4. Starts the driver & distributes application files
 - **Important Options:**
 - `--master`: defines the cluster manager
 - `--deploy-mode`: client (default) or cluster
 - `--conf`: to set key=value Spark configs
 - Java/Scala apps need `--class` to define the main class
 - Application files: `.jar`, `.py`, `.zip`, `.egg` (for dependencies)
 - **Examples:**
 - **Scala App:**

```
spark-submit --master yarn --class org.apache.spark.examples.SparkPi spark-examples.jar 10
```
 - **Python App:**

```
spark-submit --master spark://host:7077 spark_pi.py
```
 - **Managing Dependencies:**
 - **Java/Scala:** use an *uber-JAR* with all dependencies
 - **Python:**
 - Ensure identical library versions across nodes
 - Use `--py-files` for `.py`, `.zip`, `.egg`
 - Use virtual environments for isolation
 - **Spark Shell:**
 - Available in Scala & Python
 - Automatically initializes `SparkContext` and `SparkSession`
 - Ideal for quick experiments and local/cluster testing
 - Actions typed in the shell generate jobs for the cluster
-

Examples

- **Scala Shell Code Example:**
 - `val df = spark.range(0, 10)`
 - `val transformed = df.withColumn("mod", $"id" % 2)`
 - `transformed.show(4)`
 - Creates a DataFrame from 0 to 9
 - Adds a column with modulo 2
 - Shows first 4 rows
-

Key Takeaways

- `spark-submit` is the **standardized way** to run Spark apps on any cluster manager
 - It provides flexibility in config options for **deployment, resource management, and dependencies**
 - Use **uber-JARs** for Java/Scala and `--py-files` for Python to manage dependencies
 - Spark Shell gives a fast way to experiment with Spark jobs interactively
-

Lesson's Summary:

- Spark architecture has driver and executor processes coordinated by the Spark Context in the driver.
- The driver creates jobs, and the Spark Context splits jobs into tasks that can be run in parallel in the executors on the cluster. Stages are a set of tasks that are separated by a data shuffle.
- Shuffles are costly, as they require data serialization, disk, and network I/O. The driver program can be run in either client mode (connecting the driver outside the cluster) or cluster mode (running the driver in the cluster).
- Cluster managers acquire resources and run as an abstracted service outside the application. Spark can run on Spark Standalone, Apache Hadoop YARN, Apache Mesos, or Kubernetes cluster managers with specific set-up requirements.
- Choosing a cluster manager depends on your data ecosystem and factors such as ease of configuration, portability, deployment, or data partitioning needs.

- Spark can also run using local mode, which is useful for testing or debugging an application.
 - “Spark-submit” is a unified interface to submit the Spark application, no matter the cluster manager or application language.
 - Mandatory options include telling Spark which cluster manager to connect to; other options set driver deploy mode or executor resourcing.
 - To manage dependencies, application projects or libraries must be accessible for driver and executor processes, for example, by creating a Java or Scala uber-JAR.
 - Spark Shell simplifies working with data by automatically initializing the SparkContext and SparkSession variables and providing Spark API access.
-

Spark Runtime Environments

Lesson 1: Spark Environments - Overview and Options

Objectives

- Understand the **different Spark deployment environments**: Local, On-Premises, and Cloud.
 - Identify **when and why** to use each option depending on use case, scale, and budget.
 - Explore **cloud-managed Spark solutions** like IBM Cloud, Azure HDInsight, AWS EMR, and Databricks.
-

✂ Key Concepts / Steps

1. Local Machine

- **Use Case:** Development, testing, small datasets, and learning.
- **Pros:** Easy to set up, good for prototyping.
- **Cons:** Not scalable, limited by local machine resources.

2. On-Premises Cluster

- **Use Case:** Strict data compliance, high control over resources, long-term infrastructure.
- **Pros:** Full hardware control, data security.
- **Cons:** High setup/maintenance cost, limited scalability.

3. Cloud Environments

- Offers **scalability**, **managed services**, and **easy integration** with other cloud tools.

◆ IBM Cloud

- **Platform:** IBM Cloud Pak for Data
- **Use When:** You're already in IBM's ecosystem or need hybrid cloud setups.

◆ Azure HDInsight

- **Platform:** Microsoft Azure
- **Use When:** You're using Microsoft tools and need managed services.

◆ AWS EMR

- **Platform:** Amazon Web Services
- **Use When:** You need scalable, pay-as-you-go Spark clusters.

◆ Databricks

- **Platform:** Unified analytics with Spark at its core
- **Use When:** You want easy collaboration, managed infrastructure, and built-in ML tools.

Examples

- A **data scientist** might use **Databricks** for machine learning collaboration.
- A **developer** could test a Spark script locally before moving it to **AWS EMR** for large-scale processing.
- A **finance firm** with strict security requirements might run Spark on an **on-premises cluster**.

Key Takeaways

- Spark can be deployed in **local**, **on-premises**, or **cloud** environments — each with trade-offs.
 - **Cloud options** (IBM, Azure, AWS, Databricks) offer **flexibility and scalability**, especially for production workloads.
 - Choose based on **data scale**, **compliance needs**, **budget**, and **team collaboration style**.
 - **Managed services** reduce operational complexity, letting you focus on analysis instead of infrastructure.
-

Lesson 2: Using Apache Spark on IBM Cloud

Objectives

-  List the **benefits** of using Apache Spark on **IBM Cloud**
 -  Define **AIOps** and explain Spark's role in it
 -  Describe Spark integration with:
 - **IBM Spectrum Conductor**
 - **IBM Watson**
 - **IBM Analytics Engine**
-

Why Use Spark on IBM Cloud?

- **Local clusters** can be hard to configure; node visibility and communication issues arise.
 - **IBM Cloud** simplifies Spark deployment with **pre-configured setups**, enabling **easy scaling** and **quick resource allocation**.
 - Comes with **enterprise-grade security** and integrates seamlessly with IBM's AI & big data ecosystem.
-

What is AIOps?

AIOps = AI + IT Operations

- Uses **artificial intelligence** to automate or enhance IT operations.
- Helps with:
 - Aggregating massive volumes of operations data
 - Detecting patterns and significant events in complex systems
 - Diagnosing root causes of failures quickly
 - Automating responses or fixes

Spark's Role in AIOps:

- Processes huge infrastructure datasets
- Applies machine learning models to **predict, detect, or resolve** operational issues

Example: IBM Cloud Pak for Watson AIOps uses Spark to:

- Correlate operational data
 - Generate real-time alerts for availability, performance drops, or outages
-

⌘ IBM Services that Enhance Spark

1. IBM Spectrum Conductor

- A **multi-tenant platform** to deploy and manage Spark with other frameworks.
- Allows **multiple Spark apps and versions** on a **shared cluster**.
- Benefits:
 - Dynamic resource allocation
 - No downtime
 - Enterprise-level security

2. IBM Watson

- Provides **production-ready environments** for machine learning with Spark.
- Handles setup & maintenance → you focus on **model training** and **ML tasks**.
- Enables full **AI workflow integration**.

3. IBM Analytics Engine

- Built on Apache Hadoop framework.
- **Separates storage & compute** (data in IBM Cloud Object Storage).
- You only **run compute nodes when needed** → cost-effective & scalable.

🔑 Key Takeaways

- **IBM Cloud + Spark** = secure, scalable, enterprise-ready big data processing.
- **AIOps** relies on Spark for high-volume analytics and intelligent automation.
- **Spectrum Conductor** enhances Spark deployment across shared clusters.
- **Watson** supports advanced AI/ML workflows with Spark.
- **Analytics Engine** enables flexible compute/storage separation for efficient processing.

📘 Lesson 3: How to Set Up Your Own Spark Environments

🧠 Objectives

By the end of this lesson, you'll be able to:

- Set up Apache Spark on your local machine.
 - Launch and monitor Spark applications using Scala or Python.
 - Write and run basic Spark applications using DataFrames.
-

⌘ Key Concepts / Steps

1. Prerequisites:

- ☒ **Java:** Spark requires Java 8 or later. Use `java -version` to check.
- ☒ **Python (optional):** Spark has Python support via PySpark.
- ☒ **Hadoop (optional):** Spark can use HDFS but it's not needed for local development.

2. Download Spark:

- Go to spark.apache.org/downloads.
- Choose a pre-built version matching your preferred Hadoop & Scala version.

3. Set Up Environment Variables:

- `SPARK_HOME`: Points to your Spark folder.
- Add `%SPARK_HOME%/bin` to your `PATH`.

4. Configuration (Optional):

- Edit files in the `conf` directory (e.g., `spark-defaults.conf`) if needed.
- For local dev, defaults usually work fine.

5. Start Spark:

- **Interactive Shell:**
 - Scala → `spark-shell`
 - Python → `pyspark`
- **Submit Apps:**
 - Scala → `spark-submit --class <main-class> --master local <path-to-jar>`
 - Python → `spark-submit --master local <path-to-python-script>`

6. Write a Spark App (Python example):

```
from pyspark.sql import SparkSession

# Start a Spark session
spark = SparkSession.builder.appName("MySparkApp").getOrCreate()

# Load data
data = spark.read.csv("data.csv", header=True, inferSchema=True)

# Aggregate
result = data.groupBy("Name").agg({"Score": "sum"})

# Display result
result.show()

# Stop session
```

```
spark.stop()
```

7. Monitoring:

- Spark UI available at <http://localhost:4040> while an app is running.

8. Cleanup:

- Always stop the Spark session with `spark.stop()` to free resources.
-

Example

Data Sample (data.csv):

Name Score

A	10
B	15
A	20
B	5
A	30

Output after aggregation:

```
+-----+-----+
|Name |sum(Score)|
+-----+-----+
|A     |60         |
|B     |20         |
+-----+-----+
```

Key Takeaways

- You don't need a full Hadoop setup to start with Spark locally.
 - Java is a must; Python is optional but beginner-friendly via PySpark.
 - Spark has powerful built-in tools (like the web UI and SparkSession) for local development.
 - Monitoring and cleanup are important for resource management.
 - Spark apps can be written in Python using simple and intuitive syntax with DataFrames.
-

Lesson 4: Setting Apache Spark Configuration

Objectives

- Describe the **three types** of Apache Spark configuration.
 - Explain common use cases and options for each configuration method.
 - Understand when to use **static vs dynamic** configuration settings.
-

Key Concepts / Steps

Configuration Methods:

1. **Spark Properties**
 - Control application behavior.
 - Can be set:
 - **Programmatically** via `SparkSession` or `SparkConf`.
 - In `spark-defaults.conf` file.
 - Via `spark-submit` using `--conf` or `--master`.
 2. **Environment Variables**
 - Set per machine using `conf/spark-env.sh`.
 - Useful when nodes have **different setups** (hardware/software).
 - Example: `PYSPARK_PYTHON` to define which Python interpreter to use.
 3. **Logging Configuration**
 - Controlled via **Log4j** (`conf/log4j.properties`).
 - Manages **log levels** (e.g., info, debug, error) and destinations.
-

Precedence Order for Spark Properties:

1. **Programmatic** settings (highest priority).
 2. `spark-submit` options.
 3. `spark-defaults.conf` file (lowest priority).
-

Static vs Dynamic Configuration:

- **Static Configuration:**
 - Set *inside the code*.
 - For values that don't change often (e.g., app name).
 - Example:

```
.appName("MyApp").config("spark.driver.maxResultSize", "2g")
```
- **Dynamic Configuration:**

- Set *externally* (not hardcoded).
 - For **cluster-dependent** values (e.g., executor memory, cores, master URL).
 - Enables flexible tuning without modifying code.
-

Examples

Setting via `spark-submit`:

```
spark-submit --master yarn --conf spark.executor.memory=4g app.py
```

Environment Variable:

```
export PYSPARK_PYTHON=/usr/bin/python3
```

Logging:

Edit `conf/log4j.properties`:

```
log4j.rootCategory=INFO, console
```

Key Takeaways

- Apache Spark offers **flexible configuration** through properties, environment variables, and logging.
 - Choose **static config** for fixed values; use **dynamic config** for deployment-dependent settings.
 - Understanding configuration **precedence** helps avoid unexpected overrides.
 - Log management through **Log4j** ensures better debugging and monitoring.
-

Lesson 5: Running Apache Spark on Kubernetes

Objectives

After this lesson, you'll be able to:

- Describe **what Kubernetes is** and **why it's used**
 - Explain how to run Kubernetes **locally** vs. **on the cloud**
 - Understand **how to run Apache Spark** on Kubernetes
-

⌘ Key Concepts / Steps

- **What is Kubernetes (K8s)?**
 - An open-source platform for **orchestrating containerized applications** across a cluster
 - **Scalable, portable**, and highly **resilient**—can run on-prem or in the cloud
 - Perfect for **managing distributed systems** like Apache Spark
- **Why use Kubernetes with Spark?**
 - Improves **flexibility and resiliency**
 - Supports **containerized Spark apps** that are portable and isolated
 - Allows **resource sharing and better dependency management**
- **Running Kubernetes Locally**
 - Tools like **Minikube** enable local clusters for testing
 - Benefit: Test changes locally before pushing to production
 - Consistency: Containers behave the same locally and on the cluster
- **Running Kubernetes in the Cloud**
 - Ideal for production environments
 - Two options:
 - Use **bootstrap tools** to set it up manually
 - Use a **certified Kubernetes provider** (turnkey solution) for faster deployment
- **Spark on Kubernetes (v2.3+)**
 - Run Spark apps using Kubernetes as the **deployment backend**
 - Submit Spark jobs with `--master` pointing to Kubernetes API
 - Spark **Driver and Executors** run as Kubernetes **Pods**
 - Modes:
 - **Cluster mode:** Driver runs inside the cluster
 - **Client mode:** Driver runs externally, so executors need to connect to it via pod name
 - Set: `spark.kubernetes.driver.pod.name`
 - Benefit: Auto-cleanup of pods after job completion

Examples

```
# Sample Spark submit to Kubernetes
spark-submit \
  --master k8s://https://<k8s-api-server>:<port> \
  --deploy-mode cluster \
  --name spark-pi \
  --class org.apache.spark.examples.SparkPi \
  local:///opt/spark/examples/jars/spark-examples_2.12-3.0.1.jar
```

Key Takeaways

- **Kubernetes** enables robust orchestration of Spark applications with improved scalability and fault tolerance

- You can run Kubernetes **locally** for development or **on cloud** for production
 - **Spark on Kubernetes** enhances portability, resource isolation, and simplifies dependency handling
 - **Client mode** requires careful network configuration, while **cluster mode** is more hands-off
 - Kubernetes is a strong alternative to traditional Spark cluster managers like YARN or Mesos
-

Lesson's Summary:

- Running Spark on IBM Cloud provides enterprise security and easily ties in IBM big data solutions for AIOps, IBM Watson, and IBM Analytics Engine.
- Spark's big data processing capabilities work well with AIOps tools, using machine learning to identify events or patterns and help report or fix issues.
- IBM Spectrum Conductor manages and deploys Spark resources dynamically on a single cluster and provides enterprise security.
- IBM Watson helps you focus on Spark's machine learning capabilities by creating automated production-ready environments for AI.
- IBM Analytics Engine separates storage and compute to create a scalable analytics solution alongside Spark's data processing capabilities.
- You can set Spark configuration using properties (to control application behavior), environment variables (to adjust settings on a per-machine basis), or logging properties (to control logging outputs).
- Spark property configuration follows a precedence order, with the highest being configuration set programmatically, then spark-submit configuration, and lastly, configuration set in the "spark-defaults.conf" file.
- Use Static configuration options for values that don't change from run to run or properties related to the application, such as the application name.
- Use dynamic configuration options for values that change or need tuning when deployed, such as master location, executor memory, or core settings.
- Use Kubernetes to run containerized applications on a cluster to manage distributed systems such as Spark with more flexibility and resilience.

- You can run Kubernetes as a deployment environment, which is useful for trying out changes before deploying to clusters in the cloud. Kubernetes can be hosted on private or hybrid clouds and set up using existing tools to bootstrap clusters or using turnkey options from certified providers.
- While you can use Kubernetes with Spark launched either in client or cluster mode, when using Client mode, executors must be able to connect with the driver, and pod cleanup settings are required.

Module 5 Cheat Sheet: Development and Runtime Environment Options

Package/Method	Description	Code Example
<code>SparkSession.builder.getOrCreate()</code>	Creates or gets existing SparkSession.	<pre>from pyspark.sql import SparkSession spark = SparkSession.builder.appName("myApp").getOrCreate()</pre>
<code> cd </code>	Changes current directory in your terminal.	<pre>cd /usr/local/folder1 # go to folder cd - # go back to previous cd .. # move one level up</pre>
<code> docker-compose </code>	Define and run multi-container apps via YAML.	<pre># docker-compose.yml version: '3' services: web: image: nginx:latest ports: - "80:80" db: image: postgres:latest</pre>
<code> git clone </code>	Clone a repo from a URL.	<pre>git clone https://github.com/user/repo.git</pre>
<code> import </code>	Import external modules in Python.	<pre>from pyspark.sql import SparkSession</pre>
<code> pip / pip install </code>	View installed packages or install new ones.	

```
pip list
pip install pandas
```

| `pip3 install` | Install Python 3 packages. |

```
pip3 install pandas
```

| `print()` | Prints to stdout. |

```
print("Hello, World!")
```

| `python3` | Python 3 command-line tools. |

```
sudo apt-get update
sudo apt-get install python3
python3 --version
```

| `sc.setLogLevel()` | Sets Spark logging level. |

```
from pyspark import SparkContext
sc = SparkContext("local", "App")
sc.setLogLevel("INFO")
```

| `setMaster()` | Defines Spark cluster or local execution. |

```
from pyspark import SparkContext
sc = SparkContext("local[*]", "MyApp")
```

| `show()` | Displays DataFrame content. |

```
df.show()
```

| `source` | Executes a script in the current shell. |

```
source myscript.sh
```

| `virtualenv` | Creates an isolated Python environment. |

```
virtualenv myenv
```



Quick Tips

-  Use **SparkSession** instead of `SparkContext` directly when working with structured data.
 -  Use **docker-compose** to simulate multi-service environments locally.
 -  Use **virtualenv** to avoid package conflicts.
 -  Set log levels to `ERROR` or `WARN` to declutter your logs.
 -  Always use `show()` to inspect your DataFrames before transformations.
-

Module 5: Development and Runtime Environment Options

Objectives

- Understand different development and runtime environments for running Apache Spark applications.
 - Learn about Spark deployment modes (local, standalone, YARN, Mesos, Kubernetes).
 - Explore key tools and methods used in Spark environments, including containerization, configuration techniques, and resource management.
-

⚡ Key Concepts / Steps

Spark Basics & Initialization

- `SparkSession.builder.getOrCreate()`: Initializes or retrieves an existing `SparkSession`.
- `SparkContext`: The main entry point for Spark functionality; communicates with the Cluster Manager.
- `setMaster()`: Specifies execution mode (e.g., local, yarn, mesos).

Shell Commands

- `cd`: Navigate directories (`cd ..`, `cd -`, etc.).
- `source myscript.sh`: Run a script in the current shell environment.

Containerization

- **Docker Compose**: YAML-configured tool to manage multi-container applications (e.g., NGINX + Postgres).
- **Kubernetes (K8s)**: Popular container orchestration system supported natively by Spark.

Spark Configuration

- **Properties**: Configurations to tune Spark application behavior.
- **Environment variables**: e.g., `PYSPARK_PYTHON` to set Python version across nodes.
- **Logging configuration**: Uses `log4j` to control log levels.
- **Static vs Dynamic Configuration**: Hardcoded (static) vs externally modifiable (dynamic) parameters.

Development Tools

- `git clone`: Copy remote repos.
- `import`: Pull in libraries/modules in Python (e.g., `from pyspark.sql import SparkSession`).
- `pip / pip3 install`: Install Python packages from PyPI.
- `virtualenv`: Create isolated Python environments.
- `python3`: Install and run Python 3.

Spark Cluster Modes

- **Local mode**: Spark runs on a single machine (for testing/debugging).
- **Standalone cluster**: Spark's built-in cluster manager; easy to configure.
- **YARN**: Hadoop ecosystem resource manager; robust but complex.
- **Mesos**: General-purpose cluster manager with fine-grained resource sharing.
- **Kubernetes**: Orchestrates containers for scalable, cloud-native Spark workloads.

Spark Application Components

- **Driver**: Manages the `SparkContext` and schedules jobs.
- **Executors**: Perform computations and cache data.
- **Worker**: Node that launches executors.
- **Cluster Manager**: Allocates resources across the cluster.

Spark Runtime Concepts

- **Spark jobs** → split into **stages** → further split into **tasks**.
 - **Spark Shuffle**: Data re-partitioning between tasks (expensive operation).
 - **Uber-JAR**: A self-contained Java package with all dependencies.
-

Examples

Starting a `SparkSession` in Python

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.appName("myApp").getOrCreate()
```

Navigating with `cd`

```
cd /usr/local/folder1    # Move to folder
cd ..                    # Move up
cd -                      # Back to previous
```

Docker Compose Example

```
version: '3'
services:
```

```
web:
  image: nginx:latest
  ports:
    - "80:80"
db:
  image: postgres:latest
```

Installing a package

```
pip install pandas
virtualenv myenv
```

Key Takeaways

- **Spark is versatile:** It supports multiple cluster managers and can run locally for testing or across large clusters.
 - **Environment control is key:** Configuration can be static, dynamic, or set through environment variables.
 - **Containerization improves portability:** Docker and Kubernetes make Spark more scalable and manageable.
 - **Understanding architecture matters:** From drivers to executors to shuffles—knowing the Spark lifecycle helps optimize performance.
 - **You don't need to memorize everything:** But knowing what tools exist (like `spark-submit`, `git`, or `virtualenv`) makes you a more effective data engineer.
-

Module Six

Monitoring and Tuning

Introduction to Monitoring and Tuning

Lesson 1: The Apache Spark User Interface

Objectives

- Understand how to **connect to** and **navigate** the Apache Spark Web UI.
 - Learn the purpose and content of key **tabs** in the Spark interface.
-

Connecting to the Spark UI

- When a Spark application runs, the **driver program** creates a `SparkContext`.
- This triggers a **web server** to start automatically.
- Access it by visiting:
- `http://<driver-hostname>:4040`

- ⚠️ The UI is **available only while the application is running**. Once `SparkContext` stops, the web UI shuts down.
-

📁 Main UI Tabs & Their Functions

1. 🔍 Jobs Tab

- Displays a list of **all jobs** created by the application.
- Includes:
 - Job **status** (e.g., completed, failed)
 - **Duration**
 - **Number of stages**
 - **Total tasks**
 - **Description hyperlinks** → drill down into **Job Details**:
 - Visual timeline of each stage
 - Submitted time
 - Shuffle read/write stats
 - Task metrics: attempted/succeeded

2. 🎯 Stages Tab

- Lists all stages, grouped by:
 - **Active**
 - **Pending**
 - **Completed**
- Each stage has:
 - **Stage ID**
 - **Number of tasks**
 - Clickable **Stage Description** to view **Task Details**

3. 🧠 Storage Tab

- Displays **RDDs/DataFrames** that have been **cached (memory)** or **persisted (disk)**.
- For each item:
 - Shows storage **level**
 - **Size** and **number of partitions** cached

4. ⚙️ Environment Tab

- Lists technical details about:
 - Spark configuration properties
 - Environment variables
 - JVM & system properties
 - Hadoop configurations (if relevant)
 - Resource profiles

5. 🏠 Executors Tab

- Top summary of:
 - **Active/dead executors**
 - Task counts
 - Data I/O
 - **Disk/memory usage**
- Bottom list includes:
 - All executor instances (including the **driver**)
 - Links to:
 - **stdout**
 - **stderr** logs

6. 📄 SQL Tab (*Only if SQL is used*)

- Displays **SQL queries** executed by the app.
 - Click **Description** → view query **metrics**.
 - Expand to see:
 - **Query plan**
 - **Logical & physical DAGs** from the Catalyst optimizer
-

💡 Key Takeaways

- Spark UI provides **real-time visibility** into job and task execution.
 - Port **4040** is your gateway to debugging and performance insights.
 - Each tab reveals **crucial diagnostics**:
 - From high-level **job stats** to low-level **executor memory usage**.
 - SQL execution is fully traceable with **query plan visualization**.
-

📖 Lesson 2: Monitoring Application Progress

Awesome! Here's your lesson summary for “**Monitoring Application Progress**” following your favorite structure:

🧠 Objectives

By the end of this lesson, you should be able to:

- Define parts of the Spark application workflow.
- Use the Spark Application UI web server to monitor application progress.
- Describe a sample workflow by interpreting the UI.

⌘ Key Concepts / Steps

- **Why monitor?**
 - Spark apps can be long-running and prone to failure (e.g., faulty nodes).
 - Monitoring helps resolve issues quickly and prevent idle resources.
- **Spark Application UI:**
 - A centralized interface for tracking jobs, stages, and tasks.
 - Shows status, timelines, and root causes of failures.
 - Helps detect bottlenecks and performance issues.
- **Workflow Breakdown:**
 - SparkContext creates **Jobs**.
 - Jobs are broken into **Stages** (based on shuffle boundaries).
 - Stages consist of **Tasks** running on executors.
 - Stages form a **Directed Acyclic Graph (DAG)**.
 - If a task fails repeatedly, the stage and job fail.
- **Example Job Execution:**
 - **Stage 0** starts with independent tasks (no shuffle).
 - **Stage 1** starts after Stage 0 due to shuffle requirement.
 - Jobs can run sequentially or trigger others (e.g., Job 1 depends on Job 0).
- **Example Application Code:**
 - Reads a Parquet file into a DataFrame.
 - Caches, selects columns, groups by "country", and calculates mean "salary".
 - The `.collect()` action triggers execution.
- **Using the UI:**
 - **Jobs Tab:** Lists all jobs (e.g., reading data, performing aggregations).
 - **Job Details:** Shows stages and DAGs.
 - **Stage Details:** Visual timeline with task duration, status, and color-coded progress.
 - **Task Metrics:** Duration, shuffle data size, and executor logs.
- **Viewing the UI After Execution:**
 - Enable **event logging** (`spark.eventLog.enabled=true`).
 - Start the **Spark History Server**.
 - Access via `http://<hostname>:18080`.
 - Review completed applications and UI snapshots.

Example (from the lesson)

Workflow Summary:

1. **Job 0:** Reads a Parquet file (Stage 0 → Stage 1).
2. **Job 1:** Aggregates data (Tasks 6–9).
3. **UI Monitoring:**

- Job 0 has 2 stages (linked via shuffle).
 - View stage timelines to diagnose slow/failed tasks.
 - Tasks can be monitored for performance and failure analysis.
-

🌱 Key Takeaways

- Monitoring a Spark application is essential for diagnosing failures and optimizing performance.
 - The **Application UI** provides real-time insight into **jobs, stages, tasks**, and execution flow.
 - Tasks within a stage run **in parallel**, but stages follow **DAG dependencies**.
 - For post-execution monitoring, enable **event logging** and use the **History Server**.
 - Efficient debugging = less frustration + faster pipelines 🚀
-

📖 Lesson 3: Debugging Apache Spark Application Issues

🧠 Objectives

By the end of this lesson, you should be able to:

- Identify **common Apache Spark application issues**.
 - Debug issues using the **Spark Application UI**.
 - Locate and interpret **application log files** for deeper insights.
-

🔧 Key Concepts / Steps

🔥 Why Do Spark Apps Fail?

Running Spark on a cluster is complex—many parts = many places to break!
Common culprits include:

- ❌ **User Code:** Bad logic, syntax, or serialization issues.
- ⚙️ **System/Application Configurations:** Misconfigured settings.
- 📦 **Missing or incompatible dependencies:** Wrong library versions = 💥.
- 🚫 **Improper resource allocation:** CPU/memory starvation.
- 🌐 **Network communication issues:** Cluster nodes failing to talk.

🧠 What is User Code in Spark?

- **Driver Program:** Creates the `SparkContext`, runs in the driver process.

- **Executor Tasks:** Receive *serialized closures* (code + variables) to run in parallel.
- These closures are created by DataFrame operations (like `select`, `groupBy`, etc.) in the driver.

When & How Errors Happen:

- **Early errors** (syntax, serialization, validation): Spark halts quickly and cancels tasks.
- **Late errors:** Show up during execution on executor nodes—e.g., due to bad data or networking.
- Spark **retries** failed tasks (by default), but if all attempts fail → driver logs the error → app terminated.

Application Dependencies:

- Includes Python/Java files, JARs, data files, libraries, etc.
- Must be **available on all nodes**.
 - Pre-install, include in `spark-submit`, or pass as arguments.
-  Errors arise if:
 - A required Python lib isn't in executor's environment.
 - Different versions of a library exist across executors.

Resource Allocation Issues:

- Tasks wait in queue if there's not enough memory or CPU.
- If resource requests exceed what's available → tasks timeout.
- Long-running (or stuck) tasks block other tasks = .

Examples (from the lesson)

Failure Scenario:

- A Python script using a third-party library is submitted via `spark-submit`.
- The executor doesn't have the library installed → task fails.
- Another executor has an old version → API mismatch = inconsistent results.

Timeout Scenario:

- An app requests 8 cores, but cluster can only provide 6.
- Tasks stay in the queue forever until Spark gives up.

✿ Key Takeaways

- Common Spark failures = **bad code, bad config, missing dependencies, bad resources, bad networking**.
 - The **Application UI** helps monitor execution and detect errors in real-time.
 - For deeper root causes, check the **event logs** and **stderr/stdout log files**.
 - Log files live under:
 - `SPARK_HOME/work/<application-id>/stdout`
 - `SPARK_HOME/work/<application-id>/stderr`
 - Each **executor and driver process** has its own logs.
 - On Spark Standalone clusters, extra logs live in:
 - `SPARK_HOME/log/`
-

📘 Lesson 4: Understanding Memory Resources

🔧 Understanding Memory Resources for Apache Spark

🎯 Learning Objectives

- Describe memory parameters in Spark.
 - Understand how Spark manages memory internally.
 - Explain the importance and implementation of data persistence (caching).
 - Set executor and driver memory via configuration.
 - Configure memory and CPU cores in Spark Standalone clusters.
-

🧠 Memory Basics in Spark

1. Memory Limits for Driver and Executor

- Each process (driver or executor) has an upper memory limit.
- This prevents full consumption of cluster memory but must be high enough for operations.
- Exceeding memory limits causes:
 - Performance issues (e.g., disk spilling)
 - `OutOfMemoryError`

2. Driver Memory

- Handles:
 - Collecting results
 - Loading data
 - Broadcasting variables
- 🚨 Large data can overwhelm driver memory — apply filters and sample the data before collecting.

Unified Memory Management

Memory Regions

- **Executor Memory (Execution):** Runs transformations and actions.
- **Storage Memory (Caching):** Holds persisted (cached) data.
- Both share the **Java Heap Space** (M), divided into:
 - M : Unified region
 - R : Reserved subregion (non-evictable storage)

Behavior

- If caching isn't used, execution can consume full memory.
- Execution can evict storage (except R) to reclaim memory.
- Storage **cannot** evict execution memory — avoids complexity.

Data Persistence (Caching)

What It Is:

- Store intermediate results to avoid recomputation.
- Improves performance, especially in iterative workloads (e.g., ML models).

Example:

```
df = spark.range(0, 1000000).withColumn("features", rand())
df.cache()
df.count() # Triggers computation and caches it
```

- Without caching: `rand()` generates new values every time.
- With caching: values computed once and reused.

Configuring Memory & Resources

1. At Submission

```
spark-submit --executor-memory 10g ...
```

- Most common approach.
- Ensures the application gets enough memory **without hogging all resources**.

2. In Spark Standalone Cluster

When starting a worker:

```
./start-worker.sh spark://<master>:7077 --memory 16g --cores 8
```

- Specify total **memory** and **cores**.
- ⚠️ Avoid overcommitting (e.g., assigning more cores than physically available).

3. Defaults

- By default, Spark uses all available memory **minus 1GiB**.
-

✅ Summary

- Spark memory is tunable per application and cluster type.
 - Execution and storage memory share a flexible region.
 - Caching (persisting) data is key to performance — especially for complex workflows.
 - Memory and core settings must be realistic to avoid contention and performance issues.
-

📘 Lesson 5: Understanding Processor Resources

🎯 Learning Objectives:

- Understand Spark memory parameters and architecture.
 - Know how to set memory for drivers and executors.
 - Learn about data persistence (caching).
 - Configure memory in Spark Standalone and at submission.
-

🧠 Key Concepts:

1. Memory Configuration in Spark:

- Spark apps run with a **memory limit** for both the **driver** and **executor**.
- If memory is **too low**, performance suffers or crashes.
- If **too high**, the app may hog cluster resources.

2. Driver vs. Executor Memory Usage:

- **Driver:** Loads data, handles result collections, and broadcasts variables.
- **Executor:** Runs tasks and may store cached data.

3. Unified Memory Management:

- **Executor memory** and **storage memory** share a unified heap region ($\mathbb{M}$).
- Cached blocks may live in a protected sub-region ($\mathbb{R}$) that **executor memory can't evict**.
- Storage memory **can be evicted** if executor memory needs it.

4. Data Caching & Persistence:

- Caching intermediate results (like DataFrames) improves performance, especially for ML workloads.
- Example:

```
df = spark.range(1000).withColumn("features", rand())
df.cache() # Mark for caching
df.count() # Trigger actual computation
```
- Caching is **lazy**, only computed on action (like `count()`).

5. How to Set Memory:

- In Spark submit:
 - `--executor-memory 10G`
- In Spark Standalone cluster:
 - Specify total memory per worker.
 - Avoid exceeding machine capacity.
 - Default: use all memory minus 1 GiB.

Understanding Processor Resources in Apache Spark

Learning Objectives:

- Learn how Spark assigns and uses CPU cores.
- Configure cores for applications and cluster.

Key Concepts:

1. CPU Cores in Spark:

- Both **driver** and **executors** get CPU cores.
- Executors can run **tasks in parallel**, up to the number of cores assigned.

2. Task Scheduling:

- Tasks wait in a **scheduling queue** if no cores are free.
- Spark tries to **maximize parallelism** by assigning ready tasks to available executors.

3. Core Configuration Options:

- Submit with specific executor cores:
- `--executor-cores 4`
- Total executor cores across the cluster:
- `--total-executor-cores 16`
- In standalone mode, set cores for each worker:
- `./start-slave.sh spark://master:7077 --cores 4`

4. Practical Scenario (6-core node):

- App A requests 4 cores → runs first.
- App B requests 4 cores → must wait until 4 are free.
- If App B had requested 2 cores → it would start immediately (but run slower).

5. Best Practices:

- Avoid over-allocating cores or memory beyond physical capacity.
- Reserve CPU for system/master tasks.
- Tuning is required for optimal performance.

Summary Takeaways:

Resource	Key Point
Memory	Shared between execution & storage; cache wisely.
CPU Cores	More cores = more parallelism; configure carefully.
Driver	Handles collections, variable broadcast, can be overwhelmed by large datasets.
Executor	Runs tasks and handles cached data.
Caching	Speeds up repeated access, essential in ML workflows.
Cluster Configuration	Must match physical resources; use <code>--executor-memory</code> , <code>--executor-cores</code> , etc.

Lesson 6: Batch Data Ingestion Methods

◆ 1. File-Based Ingestion

What: Load data from file formats into Spark.

Why: Handle diverse data stored in CSV, JSON, Parquet, etc.

Techniques:

```
df_csv      = spark.read.csv("file.csv", header=True)
df_json     = spark.read.json("file.json")
df_parquet  = spark.read.parquet("file.parquet")
```

```
df_xml =  
spark.read.format("com.databricks.spark.xml").option("rowTag",  
"record").load("file.xml")
```

◆ 2. Database Replication & ETL

Why: Enables continuous data flow and transformation.

Techniques:

```
# Replication  
df_db.write.mode("append").saveAsTable("database_table")  
  
# ETL transformation  
df_transformed = df_raw.select("col1", "col2").withColumn("new_col",  
expr("col1 + col2"))  
df_transformed.write.mode("append").saveAsTable("transformed_table")
```

◆ 3. Data via APIs

Why: Ingest external data through RESTful APIs.

```
import requests  
response = requests.get("https://api.example.com/data")  
json_data = response.json()  
df_api = spark.read.json(spark.sparkContext.parallelize([json_data]))
```

◆ 4. Data Transfer Protocols

Why: Handle ingestion over FTP, SFTP, HTTP.

```
# FTP  
spark.read.format("com.springml.spark.sftp").option("host",  
"ftp.example.com").load("/path/to/file.csv")  
  
# HTTP  
spark.read.text("http://example.com/data.txt")
```

◆ Handling Data from Diverse Sources

☑ 1. Data Source Assessment

- Characteristics:

```
df.describe().show()
```

- Quality Check:

```
df.selectExpr("count(distinct *) as unique_records").show()
```

☑ 2. Schema Mapping & Transformation

```
df_mapped      = df.selectExpr("col1 as new_col1", "col2 as  
new_col2")  
df_transformed = df_mapped.withColumn("new_col3", expr("new_col1 +  
new_col2"))
```

☑ 3. Validation & Cleansing

```
df_validated = df.filter("col1 IS NOT NULL AND col2 > 0")  
df_cleaned   = df.na.fill(0, ["col1"])
```

☑ 4. Deduplication

```
df_deduped = df.dropDuplicates(["col1", "col2"])
```

☑ 5. Handling Unstructured Data

```
df_text = spark.read.text("text_file.txt") # Use Spark NLP, OCR,  
etc.
```

☑ 6. Governance & Compliance

```
df_gdpr = df.filter("country IN ('EU')") # Example GDPR filter
```

Summary:

- Spark supports versatile batch ingestion methods: files, databases, APIs, and protocols.
- Proper handling of schema, validation, and governance is key to reliable pipelines.
- Always ensure **scalability, data integrity, and compliance**.

Lesson's Summary:

- The Spark application UI centralizes critical information, including status information, into the Jobs, Stages, Storage, Environment, and Executors tabbed regions.
- You can quickly identify failures and then drill down to the lowest levels of the application to discover their root causes. If the application runs SQL queries, select the SQL tab and the description hyperlink to display the query's details.

- The Spark application workflow includes jobs created by the Spark Context in the driver program, jobs in progress running as tasks in the executors, and completed jobs transferring results back to the driver or writing to disk.
 - Common reasons for application failure on a cluster include user code, system and application configurations, missing dependencies, improper resource allocation, and network communications.
 - User code-specific errors include syntax, serialization, and data validation. Related errors can happen outside the code. If a task fails due to an error, Spark can attempt to rerun tasks for a set number of retries.
 - If all attempts to run a task fail, Spark reports an error to the driver and terminates the application. The cause of an application failure can usually be found in the driver event log.
 - Spark enables configurable memory for executor and driver processes. Executor memory and storage memory share a region that can be tuned.
 - Setting data persistence by caching data is one technique used to improve application performance.
 - Spark assigns processor cores to driver and executor processes during application processing. Executors process tasks in parallel according to the number of cores available or as assigned by the application.
 - You can apply the argument `--executor-cores 8` to set executor cores on submit *per executor*. This example specifies eight cores.
 - You can specify the executor cores for a Spark standalone cluster *for the application* using the argument `--total-executor-cores 50` followed by the number of cores for the application. This example specifies 50 cores.
 - When starting a worker manually in a Spark standalone cluster, you can specify the number of cores the application uses by using the argument `--cores` followed by the number of cores. Spark's default behavior is to use all available cores.
-

Module 6 Cheat Sheet: Monitoring and Tuning

Spark & Python Monitoring/Tuning Essentials

◆ **agg()**

Aggregate values like `count`, `sum`, `avg`, etc., for each group.

```
agg_df = df.groupBy("column_name").agg({"column_to_aggregate":  
"sum"})
```

◆ **cache()**

Caches a DataFrame/RDD in memory for faster access on repeated actions.

```
df = spark.read.csv("customer.csv")  
df.cache()
```

◆ **cd (Change Directory)**

Navigates between folders in the shell.

```
cd /usr/local/folder1      # Move to folder1  
cd -                       # Go back to previous directory  
cd ..                      # Go one level up
```

◆ **def + return**

Defines a function and returns output.

```
def add_numbers(a, b):  
    return a + b  
  
sum_result = add_numbers(5, 6)  
print("The sum is:", sum_result)
```

◆ **docker exec**

Run a command in a running Docker container.

```
docker exec -it my_container /bin/bash
```

◆ **docker rm**

Remove one or more containers.

```
docker rm container_id
docker rm $(docker ps -aq) # Remove all stopped containers
```

◆ **docker run**

Run a command in a new container from an image.

```
docker run [OPTIONS] IMAGE [COMMAND]
```

◆ **for loop**

Loop through lists and perform actions.

```
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print(f"I like {fruit}s")
```

◆ **groupby() (Pandas)**

Group identical data and perform aggregates.

```
grouped = df.groupby('Category').agg({
    'Value': ['count', 'sum', 'mean', 'min', 'max']
})
print(grouped)
```

◆ **repartition()**

Change the number of partitions in a DataFrame.

```
df = spark.createDataFrame(data, columns)
print(df.rdd.getNumPartitions())
df = df.repartition(2)
print(df.rdd.getNumPartitions())
```

◆ **show()**

Display rows of a DataFrame in tabular format.

```
df.show()
```

◆ `spark.read.csv("path")`

Read a CSV into a Spark DataFrame.

```
df = spark.read.csv("path.csv", header=True, inferSchema=True)
df.show()
```

◆ `wget`

Download files from the web (non-interactive).

```
wget http://example.com/file.txt
wget -b http://example.com/archive.zip # In background
```

◆ `withColumn()`

Create or modify columns in Spark DataFrame.

```
from pyspark.sql.functions import col, when

updated_df = df.withColumn("DoubleAge", col("Age") * 2) \
                .withColumn("AgeGroup", when(col("Age") <= 30,
"Young")
                                     .when((col("Age") > 30) & (col("Age") <=
40), "Middle-aged")
                                     .otherwise("Old"))
updated_df.show()
```

☒ **Tip:** Always remember to **stop the SparkSession** when done:

```
spark.stop()
```

📖 Module 6: Monitoring and Tuning

🧠 Objectives

- Understand how Spark monitors and manages resource usage through its UI and internal architecture.
 - Learn key concepts for tuning Spark performance, handling dependencies, and debugging jobs.
 - Familiarize yourself with Spark memory, persistence, parallelization, and job execution details.
-

Key Concepts / Steps

Spark UI Tabs

- **Jobs tab:** Shows a timeline and status of all jobs.
- **Stages tab:** Lists stages grouped by state (active, completed, pending).
- **Storage tab:** Displays info on cached/persisted RDDs or DataFrames.
- **Executors tab:** Memory, CPU, and I/O stats for each executor.
- **Environment tab:** Spark configs, system properties, and Hadoop settings.

Memory Management & Caching

- **Driver memory:** Holds control flow, metadata, and results.
- **Executor memory:** Processes data; caching increases usage.
- **Unified memory:** Shared by execution and storage memory.
- **Persistence:** Saves intermediate results (RDD/DataFrame) to avoid recomputation.

Data & Dependencies

- **Application dependencies:** Libraries/scripts must be accessible cluster-wide.
- **Parquet format:** Columnar, schema-preserving format for efficient I/O.
- **Serialization:** Ensures safe access to shared resources between tasks.

Execution & Monitoring

- **Directed Acyclic Graph (DAG):** Execution plan Spark builds for tasks.
- **SparkContext:** Starts the app and web UI (usually on port 4040).
- **Application ID:** Unique ID for each Spark application.
- **Multiple related jobs:** Result from multiple actions or data sources.

Parallelization

- Uses multi-threading and multiple processors to execute jobs concurrently.
- Controlled by environment variables and library-level configs.

Spark in Action

- **User code:** Driver program + serialized tasks for executors.
 - **Workflows:** Jobs → Tasks → Results back to driver or written to disk.
 - **Spark standalone:** Uses available cores; high-availability via ZooKeeper.
-



Examples (Implicit in Concepts)

- Viewing executor logs via the **Executors tab**.
 - Detecting memory overflow using the **Storage tab**.
 - Debugging job execution order through **DAG visualization**.
 - Checking for out-of-memory errors caused by excessive **caching**.
-



Key Takeaways

- Spark's UI is a powerful tool for monitoring memory usage, job status, and system health.
 - Proper configuration of memory (driver/executor) is essential to avoid crashes or timeouts.
 - DAGs, persistence, and serialization help optimize and control execution flow.
 - Understanding tabs like **Jobs**, **Stages**, **Executors**, and **Storage** is key to diagnosing performance issues.
 - Efficient resource allocation and dependency management are critical for running Spark applications smoothly.
-